CEH v13 AI (312-50) Study Guide - Module 08: Sniffing

EC-Council CEH v13 AI (312-50) Exam Prep

Study guide built from official module screenshots

Table of Contents

1. Module at a Glance
2. Big Picture Map
3. Topic-by-Topic Guide
   - 3.1 Module Title and Learning Objectives [page 1]
   - 3.2 Network Sniffing and Promiscuous Mode [page 1]
   - 3.3 Types of Sniffing: Passive and Active [page 2]
   - 3.4 Sniffing at the Data Link Layer of the OSI Model [page 2]
   - 3.5 SPAN Port [page 3]
   - 3.6 MAC Address and the CAM Table [page 4]
   - 3.7 What Happens When a CAM Table Is Full [page 4]
   - 3.8 MAC Flooding [page 5]
   - 3.9 How to Defend Against MAC Attacks [page 5]
   - 3.10 Rogue DHCP Server Attack [page 6]
   - 3.11 DHCP Starvation Attack [page 6]
   - 3.12 Defending Against DHCP Starvation and Rogue Server Attacks [page 7]
   - 3.13 ARP Spoofing Attack [page 7]
   - 3.14 ARP Spoofing and Poisoning Tools [page 8]
   - 3.15 How to Defend Against ARP Poisoning [page 8]
   - 3.16 Configuring DHCP Snooping and Dynamic ARP Inspection on Cisco Switches [page 9]
   - 3.17 ARP Spoofing Detection Tools [page 9]
   - 3.18 MAC Spoofing and Duplicating [page 10]
   - 3.19 IRDP Spoofing [page 10]
   - 3.20 VLAN Hopping [page 11]
   - 3.21 STP Attack [page 11]
   - 3.22 How to Defend Against MAC Spoofing [page 12]
   - 3.23 How to Defend Against VLAN Hopping [page 12]
   - 3.24 How to Defend Against STP Attacks [page 13]
   - 3.25 DNS Poisoning Techniques [page 13]
   - 3.26 Intranet DNS Spoofing [page 14]
   - 3.27 Internet DNS Spoofing [page 14]
   - 3.28 Proxy Server DNS Poisoning [page 15]
   - 3.29 DNS Cache Poisoning [page 15]
   - 3.30 How to Defend Against DNS Spoofing [page 16]
   - 3.31 Display Filters in Wireshark [page 16]

3.32 Sniffing Tools [page 17]
3.33 How to Defend Against Sniffing [pages 17 and 18]
3.34 How to Detect Sniffing [page 18]
3.35 Sniffer Detection Techniques: Ping Method and DNS Method [page 19]
3.36 Sniffer Detection Techniques: ARP Method [page 19]
4. Tool Encyclopedia
4.1 Attack and flooding tools
4.2 Detection, analysis, and defense tools
4.3 Look-alike tools and how to pick
5. Command Sheet
5.1 Cisco IOS: Port Security (MAC flooding and DHCP starvation defense)
5.2 Cisco IOS: DHCP Snooping
5.3 Cisco IOS: Dynamic ARP Inspection (DAI)
5.4 Cisco IOS: BPDU Guard, Root Guard, Loop Guard, and UDLD (STP defense)
5.5 Wireshark Display Filters
6. Ports, Protocols, Standards and Acronyms
6.1 Ports and protocols seen in the screenshots
6.2 Standards and acronyms
7. Attack vs Countermeasure and Look-alike Comparisons
7.1 Attack and its countermeasure
7.2 Look-alike pairs
8. Exam Cheat Sheet
8.1 Keyword to answer
8.2 Must-memorize lists
8.3 Numbers and defaults
8.4 Rules for picking the BEST option
9. Practice Questions
10. Flashcards
11. Glossary
12. Coverage Report
12.1 Map of pages to guide sections
12.2 Items marked [UNCLEAR]
12.3 Items to double check
12.4 Coverage confirmation

Source: 19 pages of screenshots (37 slides, several repeated) from the official Module 08 theory videos. Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small or blurry to read with full confidence. The guide follows the screenshots in page order so it lines up with the video.

1. Module at a Glance

Module: 08 - Sniffing (EC-Council CEH v13 AI).

What this module is (5 lines):

1. Sniffing is capturing network traffic with a packet sniffer, either passively (hub networks) or actively (switch networks, by attacking the switch, DHCP server, ARP, STP, or DNS).
2. Most of the module is attacks that force a switched network to leak its traffic: MAC flooding, DHCP starvation and rogue DHCP, ARP spoofing, MAC spoofing, IRDP spoofing, VLAN hopping, and STP attacks.
3. A second block covers DNS poisoning in its four forms: intranet, internet, proxy server, and cache poisoning.
4. Every attack has a matching Cisco IOS defense: port security, DHCP snooping, Dynamic ARP Inspection (DAI), IP Source Guard, BPDU Guard, Root Guard, Loop Guard, and UDLD.
5. The module ends with sniffing tools (Capsa, OmniPeek, Wireshark filters), general anti-sniffing countermeasures, and how to detect a sniffer on the network (ping, DNS, and ARP detection methods).

Why it matters: Sniffing is how an attacker who already has a foothold on the LAN turns "being on the network" into "seeing everyone's traffic". In the five hacking phases, sniffing mainly supports gaining access (stealing credentials in cleartext) and maintaining access (watching traffic for new opportunities). The exam leans heavily on this module because it has many named attacks that look alike (ARP spoofing vs MAC spoofing vs IRDP spoofing vs DNS spoofing) and many Cisco IOS commands that are easy to mix up.

What the exam usually asks from it:

- Passive versus active sniffing, and why switches changed the game.
- Which attack matches a description: MAC flooding, DHCP starvation, rogue DHCP, ARP spoofing, MAC spoofing, IRDP spoofing, VLAN hopping (switch spoofing vs double tagging), STP attack.
- Which defense matches which attack: port security, DHCP snooping, DAI, IP Source Guard, BPDU Guard, Root Guard, Loop Guard, UDLD, DNSSEC.

- Reading Cisco IOS command lines and `show` command output.
- The four DNS poisoning types and how to tell them apart.
- Wireshark display filter syntax.
- Tool recall: macof, arpspoof, Ettercap, bettercap, Yersinia, Capsa, OmniPeek, Wireshark.
- How to detect a sniffer: ping method, DNS method, ARP method.

The 10 things to remember most:

1. Sniffing = capturing all packets passing a point on the network with a sniffer in promiscuous mode. Passive sniffing works on hubs. Active sniffing is needed on switches.
2. A sniffer operates at the Data Link layer of the OSI model. Once it captures there, everything above (Network up to Application) is exposed without those layers knowing it.
3. A switch learns MAC-to-port mappings in its CAM (Content Addressable Memory) table. MAC flooding (tool: macof) overflows the CAM table so the switch fails open and broadcasts like a hub.
4. DHCP has two attacks: starvation (exhaust the address pool, DoS) and rogue server (answer DHCP requests with a bogus gateway, DNS, or IP). They are often combined.
5. ARP spoofing poisons ARP caches with forged replies so traffic for one host is sent to the attacker instead. MAC spoofing instead reuses a victim's real MAC address to steal their identity on the switch port.
6. VLAN hopping has two methods: switch spoofing (rogue device negotiates a trunk) and double tagging (two stacked 802.1Q tags skip the VLAN check on the first switch).
7. An STP attack makes a rogue switch claim priority 0 so it becomes the root bridge and pulls all traffic through itself.
8. DNS poisoning has four forms: intranet (LAN, uses ARP poisoning), internet (Trojan changes the victim's DNS server), proxy server (Trojan changes the browser's proxy), and cache poisoning (forged records injected into a resolver's cache).
9. Cisco defenses map one-to-one to attacks: port security (MAC flooding, DHCP starvation), DHCP snooping (rogue DHCP, feeds DAI and IP Source Guard), Dynamic ARP Inspection (ARP and MAC spoofing), IP Source Guard (IP/MAC spoofing), BPDU Guard / Root Guard / Loop Guard / UDLD (STP attacks).
10. Detect a sniffer with the ping method (wrong MAC, right IP), the DNS method (unexpected reverse DNS lookup traffic), or the ARP method (only the sniffing host answers a probe about traffic it should not have seen).

2. Big Picture Map

```
SNIFFING (Module 08)
|
+-- CONCEPTS
|     Packet sniffing -> Promiscuous mode -> Passive vs Active
|     -> Data Link layer -> SPAN port -> CAM table
|
+-- LAYER 2 / INFRASTRUCTURE ATTACKS
|     MAC flooding -> DHCP starvation / Rogue DHCP -> ARP spoofing
|     -> MAC spoofing -> IRDP spoofing -> VLAN hopping -> STP attack
|     (each has a matching Cisco IOS defense)
|
+-- DNS POISONING
|     Intranet -> Internet -> Proxy server -> Cache poisoning
|     -> Defenses (DNSSEC and 13 more)
|
+-- TOOLS, FILTERS, DEFENSE AND DETECTION
      Wireshark filters -> Sniffing tools (Capsa, OmniPeek)
      -> General anti-sniffing countermeasures
      -> Detecting a sniffer (ping, DNS, ARP methods)
```

Read top to bottom: first understand what sniffing is and why a switch should stop it, then learn each way attackers break that protection, then learn the matching Cisco defense, then the DNS-level version of the same idea, and finally the tools and detection methods that tie it together.

3. Topic-by-Topic Guide

3.1 Module Title and Learning Objectives [page 1]

What it is (simple words): The module is "Module 08 - Sniffing". Four learning objectives: summarize sniffing concepts, demonstrate different sniffing techniques, use sniffing tools, and explain sniffing countermeasures.

Analogy: A wiretap on a phone line. First you learn how a wiretap works (concepts), then the different ways to attach one (techniques), then the devices used (tools), then how to stop someone from tapping your line (countermeasures).

How the module fits the hacking flow (ADDED CONTEXT):

1. Modules 2 to 5 gather information from outside the network.
2. Module 6 (System Hacking) and Module 7 (Malware Threats) get a foothold inside the network.
3. Module 8 (Sniffing) uses that foothold on the LAN to capture traffic and steal credentials or redirect victims.

4. Later modules build on this: Module 9 (Social Engineering) can use sniffed data, Module 11 (Session Hijacking) often starts from a sniffed session token, Module 16 (Wireless) repeats these ideas over Wi-Fi.

Real-world use: A penetration tester who has plugged a laptop into an office network switch wants to see if any application still sends passwords in cleartext. The four objectives are the exact plan: understand sniffing, pick a technique for this switched network, run a tool, and then write the countermeasures into the report.

Tools and commands used here: None. This slide is the plan.

EXAM FOCUS:

- The exam rarely quotes the objective list. It tests each block separately across many questions.
- Trap: do not confuse "sniffing" (capturing existing traffic) with "spoofing" (sending forged traffic). Many attacks in this module use spoofing as the method to enable sniffing.

Memory trick: "C-T-T-C": Concepts, Techniques, Tools, Countermeasures.

3.2 Network Sniffing and Promiscuous Mode [page 1]

What it is (simple words): Packet sniffing is monitoring and capturing all data packets passing through a given network, using a software application or hardware device. It lets an attacker observe and access the entire network traffic from a given point. A sniffer works by switching the network interface card (NIC) of a system into promiscuous mode, so it listens to all data transmitted on its segment instead of only the frames addressed to it.

Analogy: A normal NIC is like a person who only opens mail with their own name on the envelope. A NIC in promiscuous mode is like a nosy neighbor who opens every envelope that passes through the mailroom.

How it works (numbered steps):

1. The attacker places a system or a sniffer on the network segment.
2. The NIC driver is set to promiscuous mode, so it passes every frame on the wire up to the sniffer software instead of dropping frames not addressed to it.
3. On a hub, every port already receives every frame, so promiscuous mode alone is enough.
4. On a switch, the attacker must first force the switch to behave like a hub (see MAC flooding and other active techniques below), because a switch normally sends a frame only out the port that holds the destination MAC.
5. The sniffer records the captured frames for later reading.

Real-world use: An attacker connects a laptop to an open network port in a conference room. With a sniffer in promiscuous mode and a switch forced to broadcast, the attacker reads Telnet passwords, email traffic, syslog traffic, router configuration traffic, web traffic, DNS traffic, FTP passwords, chat sessions, and account information, all passing through that one point.

Tools and commands used here: Named later in section 4 and 5 (Wireshark, Capsa, OmniPeek, tcpdump as ADDED CONTEXT).

AI angle: An attacker can feed captured cleartext traffic into an AI tool to quickly summarize interesting sessions (logins, file transfers). Example prompt: "Here is a list of HTTP Basic Auth headers I captured. Decode each one and list the usernames found." A defender can use the same approach to triage packet captures faster during incident response.

Countermeasures and defense: Covered fully in sections 3.33 to 3.37. In short: use switches, encrypt traffic end to end, and check for NICs in promiscuous mode.

EXAM FOCUS:

- Keyword "promiscuous mode" -> a NIC that accepts all frames, not only ones addressed to it.
- Keyword "any data packets passing through a given network" with no modification -> sniffing, not spoofing or a man-in-the-middle attack (though sniffing is often the result of one).
- Trap: a plain sniffer on a switch (no active attack) only sees broadcast traffic and its own host's traffic, not everyone else's. The active techniques later in this module are what make switch sniffing work.

Memory trick: "Promiscuous = picks up everyone's mail, not just its own."

3.3 Types of Sniffing: Passive and Active [page 2]

What it is (simple words): Passive sniffing is sniffing through a hub. Every packet is sent to every port by design, so the attacker needs no extra steps; all hosts on a hub-based network can see all traffic, and the attacker easily captures it. Hub usage is now outdated; most modern networks use switches, so passive sniffing by itself rarely works anymore. Active sniffing is used to sniff a switch-based network. It involves injecting Address Resolution Protocol (ARP) packets into the network to flood the switch's Content Addressable Memory (CAM) table, which normally tracks host-to-port connections.

Analogy: Passive sniffing is listening in a room where everyone already talks out loud to everyone (a hub). Active sniffing is secretly jamming the room's seating chart (the switch's CAM table) so the room's speaker system starts broadcasting every private conversation to every seat.

How it works (numbered steps):

1. Passive: attacker connects to a hub-based segment and simply listens; no packets are injected.
2. Active: attacker injects crafted packets (ARP, DHCP, STP, and others) to break the switch's normal one-port-per-conversation behavior.
3. Active sniffing techniques listed on the slide: MAC Flooding, DHCP Attacks, DNS Poisoning, Switch Port Stealing, ARP Poisoning, and Spoofing Attack. Each is covered later in this guide.

Real-world use: A tester on an old factory floor network with hubs captures all traffic for free (passive). The same tester in the modern office, which uses switches, has to run arpspoof first to get the same result (active).

Tools and commands used here: macof, Yersinia, arpspoof, Ettercap, bettercap (all covered in sections 3.8, 3.11, and 3.14).

AI angle: None directly on this slide; AI tools are used later to help write and tune the attack scripts (ADDED CONTEXT, see Module 1's AI angle on scripting), not in this specific concept slide.

Countermeasures and defense: Use switches instead of hubs as a first step (removes passive sniffing), then add the active-sniffing-specific defenses in sections 3.9, 3.12, 3.16, 3.22, 3.23, and 3.24.

EXAM FOCUS:

- Keyword "hub", "no extra packets sent" -> passive sniffing.
- Keyword "switch-based network", "inject ARP", "CAM table" -> active sniffing.
- Trap: a question may say "the attacker only listens, no packets sent, network uses switches." That description is contradictory for a successful attack; on switches, listening alone usually is not enough unless the attacker is already seeing broadcast or their own traffic.
- List to memorize: the six active sniffing techniques named on the slide (MAC Flooding, DHCP Attacks, DNS Poisoning, Switch Port Stealing, ARP Poisoning, Spoofing Attack).

Memory trick: "Passive sniffs what already flows to it (hub). Active makes it flow to it (switch)." For the six active techniques: "My Dog Drools Spit Across Pillows" (Mac flooding, DHCP attacks, DNS poisoning, Switch port stealing, ARP poisoning, Spoofing).

> **ADDED CONTEXT:** "Switch Port Stealing" is named on this slide's list but no later slide in the screenshots explains it by that exact name. The closest matching concept in the module is MAC spoofing/duplicating (section 3.18), where an attacker claims another host's MAC address to "steal" its switch port mapping. Treat them as the same idea for the exam unless your official courseware draws a sharper line.

3.4 Sniffing at the Data Link Layer of the OSI Model [page 2]

What it is (simple words): Sniffers operate at the Data Link layer of the OSI model. Networking layers are designed to work independently of each other. This means that if a sniffer captures data at the Data Link layer, the upper layers (Network, Transport, Session, Presentation, Application) are never aware that the sniffing happened, because each layer only trusts the layer below it to deliver data correctly.

Analogy: A postal sorting depot (Data Link layer) is where someone secretly photocopies every letter before it is delivered. The sender, the letter writer (Application layer), and everyone else up the chain never finds out, because from their point of view the letter simply arrived.

How it works (numbered steps):

1. Data travels down the sender's OSI stack from Application to Physical.
2. It crosses the wire at the Physical layer and arrives at Data Link.
3. A sniffer with a NIC in promiscuous mode grabs frames right here, before any of the upper layers at the receiving end even process them.
4. The "Application Stream" box on the slide shows protocols such as POP3, IMAP, IM, SSL, and SSH running at the Application layer, with Protocols/Ports and IP addresses at the Network layer, all of it funneled down through the "Initial Compromise" point at the Data Link layer, and finally the Physical Links.
5. Because the compromise point is below all of them, a sniffer can read plaintext protocols (POP3, IMAP, IM) and even see encrypted protocols (SSL, SSH) travel past, though it cannot read the encrypted payload without breaking the encryption separately.

Real-world use: A SOC analyst explains to management why antivirus and application-level logging alone did not catch a sniffing attack: the attack happened a layer below where those tools look.

Tools and commands used here: Any packet sniffer (Wireshark, tcpdump); they all work at this layer by capturing raw frames.

AI angle: None on this slide.

Countermeasures and defense: Because sniffing happens below the application, the fix must also work at or below that layer: encrypt the data itself (so even a captured frame is useless) or control which frames a switch delivers (port security, DHCP snooping, DAI).

EXAM FOCUS:

- Keyword "layers work independently", "data link layer", "upper layers not aware" -> this concept.

- Trap: application-layer encryption (like login over HTTPS) still gets sniffed as encrypted bytes; the fix is not "stop sniffing at a higher layer", it is "make what is sniffed worthless" (encryption) or "stop the frames from reaching the attacker" (switch-level defenses).
- Look-alike: this is different from a Layer 3 (Network layer) attack such as IP spoofing. Sniffing itself is a Layer 2 activity even though what it captures includes data from every layer above.

Memory trick: "Sniffing lives in the basement (Data Link). Everyone upstairs keeps talking, unaware the basement is listening."

3.5 SPAN Port [page 3]

What it is (simple words): A SPAN (Switched Port Analyzer) port is a port that is configured to receive a copy of every packet that passes through a switch. When an attacker connects to the SPAN port, they can compromise the entire network, because that one port now mirrors everyone's traffic.

Analogy: A security camera feed port in a building. It is meant for the security guard (an IDS or a legitimate protocol analyzer), but anyone who plugs into that same feed port sees every hallway at once.

How it works (numbered steps):

1. A network administrator configures one switch port as a SPAN port for monitoring.
2. The switch copies every packet seen on its other ports to this SPAN port.
3. A legitimate protocol analyzer or IDS plugs into the SPAN port to watch for problems or attacks across the whole switch.
4. If an attacker instead gains access to this SPAN port (or convinces an administrator to add one, or compromises the switch management interface), the attacker now receives full visibility of the network without needing MAC flooding or ARP poisoning at all.

Real-world use: A company already runs an IDS off a SPAN port for legitimate monitoring. An insider threat plugs a second cable into the same SPAN port (or a misconfigured extra SPAN session) and now also captures everything the IDS sees, for free.

Tools and commands used here: Protocol analyzer or sniffer (Wireshark, Capsa) connected physically or virtually to the SPAN port. Cisco IOS configuration commands for SPAN are not shown on the slide (ADDED CONTEXT: typically `monitor session <n> source interface <x>` and `monitor session <n> destination interface <y>`, verify exact syntax for your IOS version).

AI angle: None on this slide.

Countermeasures and defense: Treat SPAN port configuration as a privileged change; restrict who can configure switch monitoring, audit SPAN sessions regularly, and physically secure switch access.

EXAM FOCUS:

- Keyword "port configured to receive a copy of every packet" -> SPAN port.
- Trap: a SPAN port is a legitimate network management feature, not inherently an attack. The exam question is usually about what happens if an attacker reaches it, or how it differs from a normal switch port.
- Look-alike: SPAN port (mirrors traffic by design) versus MAC flooding (forces a normal port to see traffic it should not, by breaking the CAM table).

Memory trick: "SPAN = Switch's Photocopy of All Network traffic, handed to whoever plugs into that one port."

3.6 MAC Address and the CAM Table [page 4]

What it is (simple words): Every switch has a fixed-size, dynamic Content Addressable Memory (CAM) table. This table stores information such as the MAC addresses available on each physical port, together with their associated VLAN (virtual local area network) parameters. The switch uses the CAM table to know which port to send a frame out of, instead of broadcasting everything to every port.

Analogy: A hotel front desk's room list. Each guest's name (MAC address) is written next to a room number (switch port). When a letter arrives for a guest, the front desk checks the list and delivers it only to that one room, instead of knocking on every door.

How it works (numbered steps):

1. A MAC address is 6 bytes (48 bits) long: a 3-byte Organizationally Unique Identifier (OUI, identifies the manufacturer) plus a 3-byte NIC-specific part (identifies the individual card).
2. Within the first byte, specific bits have special meaning: one bit marks whether the address is unicast (0) or multicast (1); another bit marks whether the address is globally unique (0, assigned by the manufacturer) or locally administered (1, manually set, which is how MAC spoofing works).
3. The switch watches the source MAC address of every incoming frame and records it in the CAM table together with the VLAN and the port it arrived on, a process called MAC learning.
4. The table on the slide has columns Vlan, MAC Add, Type (Dynamic, meaning learned automatically), Learn, Age, and Ports. [UNCLEAR on page 4: the exact sample MAC address hex values are hard to read; treat the sample rows only as an illustration of the column layout, not as exact data.]

5. When a frame arrives for a known destination MAC, the switch forwards it only out the matching port. Unknown destinations are still broadcast (flooded) until learned.

Real-world use: A network engineer runs the switch's MAC table command during troubleshooting to confirm which port a misbehaving device is plugged into, by matching its MAC address to a port number in the table.

> **ADDED CONTEXT:** On Cisco IOS the live command to view this table is `show mac address-table` (sometimes written as `show mac-address-table` on older IOS). The CAM table has a fixed maximum size (commonly a few thousand to tens of thousands of entries depending on the switch model); this fixed size is exactly what MAC flooding abuses next.

Tools and commands used here: `show mac address-table` (ADDED CONTEXT, verify exact syntax per platform).

AI angle: None on this slide.

Countermeasures and defense: Port security (section 3.9) directly limits how many MAC addresses a single port can register in this table.

EXAM FOCUS:

- Keyword "fixed-size dynamic table", "MAC to port mapping" -> CAM table.
- Numbers to remember: MAC address = 48 bits = 6 bytes = 3-byte OUI + 3-byte NIC-specific part.
- Trap: CAM table (switch, Layer 2, MAC-to-port) is not the same as the ARP table / ARP cache (any IP host, Layer 2-to-3, IP-to-MAC). Both get attacked in this module but by different methods.

Memory trick: "CAM = switch's phonebook: name (MAC) next to address (port)."

3.7 What Happens When a CAM Table Is Full [page 4]

What it is (simple words): Once the CAM table fills up on a switch, additional ARP request traffic floods every port on the switch. This changes the behavior of the switch: it resets to its learning mode and starts broadcasting on every port, acting like a hub. This attack also fills the CAM tables of adjacent switches, spreading the effect.

Analogy: The hotel front desk's room list (CAM table) runs out of pages. With no list left to check, the exhausted staff give up sorting mail and just shove a copy of every letter under every door in the hotel.

How it works (numbered steps):

1. An attacker (or a flooding tool) generates far more unique source MAC addresses than the CAM table can store.
2. The table fills to its maximum size.
3. The switch can no longer learn or look up new mappings reliably, so it falls back into a fail-open, flood-everything mode, much like an unmanaged hub.
4. In the example diagram: host Y is on Port 3 and host Z is on Port 3 (both recorded against the same port because the table is confused or overloaded), and traffic meant to flow only from A to B is now also visible to MAC C, an uninvolved third host, simply because the switch is flooding.
5. The flood of broadcast-like traffic can also exhaust the CAM tables on switches connected to the overloaded one.

Real-world use: During a MAC flooding attack, every host plugged into the affected switch (not just the attacker) temporarily sees traffic that was never meant for them, which is exactly the opening a sniffer needs.

Tools and commands used here: macof (section 3.8) is the classic tool used to deliberately fill the CAM table.

AI angle: None on this slide.

Countermeasures and defense: Port security with a MAC address maximum per port (section 3.9) prevents any single port from registering enough addresses to overflow the table.

EXAM FOCUS:

- Keyword "CAM table full", "switch acts like a hub", "floods every port" -> this failure mode, which is the direct goal of MAC flooding.
- Trap: this is not a vulnerability in ARP itself; it is a side effect of how switches are built to fail open rather than fail closed when their memory is exhausted.

Memory trick: "Full table, failed switch, free-for-all flood."

3.8 MAC Flooding [page 5]

What it is (simple words): MAC flooding involves flooding the CAM table with fake MAC and IP address pairs until it is full. The switch then acts as a hub by broadcasting packets to all machines, so attackers can easily sniff the traffic.

Analogy: Mailing thousands of fake name tags to the hotel front desk until the real room list is buried and the staff start sliding every letter under every door.

How it works (numbered steps):

1. The attacker runs a flooding tool on the switch segment.

2. The tool sends a huge number of frames, each with a different, often random, source MAC address (and sometimes a spoofed source IP too).
3. The switch tries to learn every one of these fake addresses into its CAM table.
4. The table fills up (section 3.7), and the switch starts flooding all frames to all ports.
5. The attacker's sniffer, now receiving everyone's traffic, captures it.

What the slide shows: The tool **macof** is named as a Unix/Linux tool that is part of the dsniff collection. macof sends random source MAC and IP addresses. The slide states this tool floods the switch's CAM table by sending a large number of bogus MAC entries [UNCLEAR on page 5: the exact number shown, something on the order of 100,000+ entries, is hard to read precisely; treat the value as "a very large number per short time" and verify the exact figure before quoting it on the exam]. A terminal screenshot shows macof running [UNCLEAR on page 5: exact command-line output].

> **ADDED CONTEXT:** macof ships as part of the `dsniff` suite (by Dug Song). Typical usage is simply `macof -i <interface>` to start flooding the local segment; add `-d <destination>` or other flags to target a specific host (verify exact flags for your installed version).

Real-world use: A tester plugs into an open switch port in a lab test and runs macof for sixty seconds. Within that time the switch's CAM table overflows and the tester's Wireshark capture starts showing other hosts' traffic that was previously invisible.

Tools and commands used here: macof (dsniff suite).

AI angle: An attacker could ask an AI assistant to explain dsniff tool options or to write a quick script that automates running macof followed by starting a capture. A defender could ask an AI assistant to explain what a sudden spike of new MAC addresses on one port means in switch logs. Example prompt (defender): "My switch log shows 50,000 new MAC learns on port Gi1/1 in one minute. What attack does this match and what Cisco command stops it?"

Countermeasures and defense: See section 3.9 (port security). In short: limit the number of MAC addresses allowed per switch port.

EXAM FOCUS:

- Keyword "flooding the CAM table with fake MAC/IP pairs" -> MAC flooding. Tool keyword "macof", "dsniff" -> MAC flooding.
- Trap: macof attacks the CAM table (Layer 2 switch memory), which is different from ARP cache poisoning (attacks a host's or switch's ARP table), even though both can end in the attacker sniffing traffic.

Memory trick: "macof = MAC Overflow: it floods, you sniff."

3.9 How to Defend Against MAC Attacks [page 5]

What it is (simple words): Port security on Cisco switches restricts inbound traffic from only a selected set of MAC addresses on a given switch port, and limits MAC flooding attacks by capping how many MAC addresses a port will ever learn.

Analogy: Giving the hotel front desk a rule: "Room 3 may only ever be assigned to this one specific guest name, forever. Reject anyone else who tries to check in under a different name on that room."

How it works (numbered steps):

1. Enable port security on the interface: `switchport port-security`.
2. Set the maximum number of MAC addresses allowed on that port: `switchport port-security maximum <1-3072>` (the valid range on the slide is 1 to 3072; for strict protection, set this to 1 so only the one expected device may ever use that port).
3. Choose what happens on a violation: `switchport port-security violation restrict` (drop violating traffic and log it, without shutting down the port entirely; other violation modes exist such as `protect` and `shutdown`, ADDED CONTEXT, verify for your IOS version).
4. Set how long a learned (secure) MAC address is kept before it can age out: `switchport port-security aging time 2`.
5. Choose the aging type: `switchport port-security aging type inactivity` (age out only if the MAC has been inactive, rather than a flat timer).
6. Optionally alert on repeated violations: `snmp-server enable traps port-security trap-rate 5`.

Real-world use: A branch office configures every access port with `switchport port-security maximum 1`. An employee's laptop is replaced by an attacker's device plugged into the same cable; the port immediately violates the MAC limit and the switch restricts or shuts the port down, alerting the help desk.

Tools and commands used here: The five port security commands above, all Cisco IOS.

AI angle: None directly on this slide; ADDED CONTEXT - an AI assistant can help turn a list of expected device MAC addresses into a bulk configuration script for many ports at once.

Countermeasures and defense: This entire section is itself the countermeasure for MAC flooding (section 3.8) and also helps limit DHCP starvation (section 3.11), because both rely on a single port being allowed to generate unlimited unique MAC addresses.

EXAM FOCUS:

- Keyword "limit MAC addresses per port" -> port security, specifically the `maximum` keyword.
- Number to remember: the valid maximum range on the slide is 1 to 3072.
- Command recall: `switchport port-security`, `switchport port-security maximum`, `switchport port-security violation restrict`, `switchport port-security aging time`, `switchport port-security aging type inactivity`.
- Trap: port security defends the switch port itself; it does not stop ARP spoofing or DNS poisoning, which need separate defenses (DAI, DNSSEC).

Memory trick: "Port security = bouncer who only lets in 1 to 3072 named guests per door, and kicks out (or just blocks) gatecrashers."

3.10 Rogue DHCP Server Attack [page 6]

What it is (simple words): The attacker sets up a rogue DHCP server on the network and responds to DHCP requests with bogus IP configuration settings, resulting in compromised network access for the victim. This attack usually works together with the DHCP starvation attack: the attacker first knocks the genuine DHCP server out of the race (by exhausting its address pool or otherwise beating it to the reply), then answers the victim's DHCP request with a rogue TCP/IP setting.

Analogy: A fake taxi dispatcher who answers your call faster than the real dispatcher and sends you a driver who works for a kidnapper, giving you wrong directions (gateway), a wrong phone book (DNS server), or a fake address (IP) on purpose.

How it works (numbered steps, the DORA exchange abused):

1. The user's machine broadcasts DHCPDISCOVER (IPv4) or a SOLICIT message (IPv6) looking for any DHCP server.
2. Normally only the genuine DHCP server answers. In this attack, a rogue server on the network also answers (or answers first), unicasting back DHCPOFFER (IPv4) or ADVERTISE (IPv6).
3. The user broadcasts DHCPREQUEST (IPv4) or REQUEST (IPv6), accepting an offer (ideally the genuine one, but the attack is designed to make the rogue offer win).
4. The rogue server unicasts DHCPACK (IPv4) or REPLY (IPv6), finalizing bogus settings onto the victim.
5. From then on, the attacker secretly listens to all the traffic passing to or from the user, because the user's traffic now flows through attacker-controlled infrastructure.

The three ways a rogue DHCP server hurts the victim:

- **Wrong default gateway:** attacker becomes the gateway, a man-in-the-middle for all outbound traffic.

- **Wrong DNS server:** attacker becomes the DNS server, able to redirect any lookup (see section 3.26).
- **Wrong IP address:** denial of service, often paired with a spoofed IP that collides with another host.

Real-world use: An attacker drops a small rogue DHCP device onto an open network jack in a lobby. New laptops that join the Wi-Fi or wired network get the attacker's gateway address and silently route all their traffic through the attacker's laptop.

Tools and commands used here: A rogue DHCP server can be stood up with tools such as `dnsmasq`, the Metasploit `dhcp_server` module, or custom scripts (ADDED CONTEXT, verify tool availability).

AI angle: None directly shown; ADDED CONTEXT - an attacker could use an AI assistant to quickly generate a rogue DHCP server configuration file with specific bogus gateway and DNS values.

Countermeasures and defense: DHCP snooping (section 3.12) is the direct fix: it only accepts DHCP server responses (DHCPOFFER/DHCPACK) on switch ports explicitly marked as trusted.

EXAM FOCUS:

- Keyword "attacker sets up a DHCP server", "bogus IP configuration" -> rogue DHCP server attack.
- Trap: a rogue DHCP server attack and a DHCP starvation attack are often paired, but they are not identical. Starvation is a denial-of- service against the legitimate DHCP server's address pool; a rogue server attack is about handing out malicious configuration. A question may ask which one actually results in the attacker becoming the gateway or DNS server (answer: rogue DHCP server).
- Memorize the three bad outcomes: wrong gateway, wrong DNS, wrong IP.

Memory trick: "Rogue DHCP = Rogue GPS: gives you the wrong turn (gateway), the wrong map book (DNS), or strands you with no address at all (IP)."

3.11 DHCP Starvation Attack [page 6]

What it is (simple words): This is a denial-of-service (DoS) attack on the DHCP server, where the attacker broadcasts forged DHCP requests and tries to lease all of the IP addresses available in the DHCP scope. Because every address is already leased out (to the attacker's fake clients), the legitimate user is unable to obtain or renew an IP address requested via DHCP, and fails to get access to the network.

Analogy: A scalper buying every single ticket to a concert the moment they go on sale, using hundreds of fake names, so no real fan can get one.

How it works (numbered steps):

1. The attacker generates a flood of DHCPDISCOVER/DHCPREQUEST messages, each using a different, usually spoofed, source MAC address.
2. The genuine DHCP server, following its normal process, offers and then leases an IP address from its scope (for example 10.10.10.1 through 10.10.10.254 in the slide's diagram) to each fake request.
3. The scope runs out of available addresses before the real users get a chance to request one.
4. Legitimate users are now denied service: no new leases, and existing leases eventually cannot be renewed either.
5. Rogue DHCP server attacks (section 3.10) are often launched right after starvation, since the genuine server is effectively disabled and has no addresses left to offer, making a rogue server's answers the only ones available.

What the slide shows: The tool **Yersinia** is named as the DHCP starvation attack tool, shown with a terminal screenshot [UNCLEAR on page 6: exact on-screen values]. A separate list of DHCP Starvation Attack Tools is given: dhcpStarvation.py (github.com), Metasploit (metasploit.com, verify exact URL), Hyenae (sourceforge.net), and DHCPig (github.com).

> **ADDED CONTEXT:** Yersinia is a multi-protocol attack framework that also supports attacks against STP, CDP, DTP, HSRP, and other Layer 2 protocols, not just DHCP. It is commonly run as `yersinia -G` for a graphical mode or `yersinia dhcp -attack <n>` from the command line (verify exact syntax for your installed version).

Real-world use: An attacker runs Yersinia's DHCP starvation attack in a branch office overnight. By morning no new employee laptop can get an IP address, and IT spends hours troubleshooting a "network outage" that is really a deliberately exhausted DHCP scope.

Tools and commands used here: Yersinia, dhcpStarvation.py, Metasploit, Hyenae, DHCPig.

AI angle: None directly shown; ADDED CONTEXT - defenders can ask an AI assistant to draft a detection rule that alerts when DHCP lease utilization crosses 90% in a short time window, a sign of starvation.

Countermeasures and defense: See section 3.12 (port security limiting MAC addresses per port, plus DHCP snooping).

EXAM FOCUS:

- Keyword "exhausts the DHCP address pool", "legitimate user cannot get an IP" -> DHCP starvation, a denial-of-service attack.
- Tool keyword "Yersinia" -> DHCP starvation (and other Layer 2 attacks).

- Trap: starvation by itself is a DoS; it does not automatically give the attacker credentials or traffic. It becomes dangerous mainly when combined with a rogue DHCP server.

Memory trick: "Starvation = sold-out scope. Every address gone before the real user arrives."

3.12 Defending Against DHCP Starvation and Rogue Server Attacks [page 7]

What it is (simple words): Two Cisco IOS features stop both DHCP attacks together. Port security defends against starvation attacks by configuring a MAC address limit on the switch's edge ports; once that limit is reached, the switch drops packets from any further new MAC addresses. DHCP snooping defends against rogue servers by making the switch only accept a DHCP server transaction (DHCPOFFER, DHCPACK) that arrives from a port explicitly marked as trusted.

Analogy: Port security limits how many fake names one scalper can use at one ticket window (stopping starvation). DHCP snooping makes the venue only honor tickets stamped by the one real ticket booth, ignoring any other booth that tries to hand out tickets (stopping a rogue server).

How it works (IOS Switch Commands, for port security on edge ports):

```
switchport port-security
switchport port-security maximum 1
switchport port-security violation restrict
switchport port-security aging time 2
switchport port-security aging type inactivity
```

How it works (IOS Global Commands, for DHCP snooping):

```
ip dhcp snooping
ip dhcp snooping vlan <id>
ip dhcp snooping trust
```

1. `ip dhcp snooping` turns DHCP snooping on globally for the switch.
2. `ip dhcp snooping vlan <id>` configures which VLANs are snooped.
3. `ip dhcp snooping trust` is applied on an interface to configure it as a trusted port (this should only be the uplink toward the real DHCP server).

> **REMEMBER:** All ports in the VLAN are untrusted by default once DHCP snooping is enabled. Only the port facing the legitimate DHCP server should be marked trusted; every access port facing end users stays untrusted, so a rogue server plugged into any of them is ignored.

Real-world use: A network team enables DHCP snooping on VLAN 10 with only the uplink to the core switch trusted. An employee plugs in a rogue DHCP device by mistake. The switch silently drops its DHCPOFFER messages because they arrive on an untrusted port, and nobody on the network is affected.

Tools and commands used here: The five commands above.

AI angle: None directly shown.

Countermeasures and defense: This entire slide is the countermeasure for sections 3.10 and 3.11.

EXAM FOCUS:

- Keyword "MAC limit on edge ports" -> port security (fixes starvation).
- Keyword "accept DHCP transaction only from a trusted port" -> DHCP snooping (fixes rogue server attacks).
- Must-memorize default: all VLAN ports are untrusted by default under DHCP snooping.
- Command recall: `ip dhcp snooping`, `ip dhcp snooping vlan <id>`, `ip dhcp snooping trust`.

Memory trick: "Snooping trusts only the real booth. Port security caps how many fake names one window can process."

3.13 ARP Spoofing Attack [page 7]

What it is (simple words): Address Resolution Protocol (ARP) is a stateless protocol used for resolving IP addresses to machine (MAC) addresses. ARP spoofing involves constructing many forged ARP request and reply packets to overload the switch. The switch is set in a "forwarding mode" after the ARP table is flooded with spoofed MAC entries, and attackers can then sniff all the network traffic. Attackers flood a target computer's ARP cache with forged entries, which is also known as ARP poisoning.

Analogy: ARP is like shouting across a room, "Who has apartment 1.1.1.1?" and anyone can shout back a fake answer claiming to be that apartment, since nobody checks ID. Whoever shouts back fastest or loudest (or simply answers unprompted) gets believed.

How it works (numbered steps, from the diagram):

1. User A needs to connect to IP 10.1.1.1 but does not have a MAC address for it yet.
2. User A sends an ARP request, which the switch broadcasts to all ports.
3. User B (the legitimate owner of 10.1.1.1) and User C both receive the broadcast request.

4. Both the legitimate host and the attacker (a malicious user) respond. The attacker oversteps the real ARP reply and sends its own spoofed reply, claiming to own 10.1.1.1 with the attacker's own MAC address.
5. User A's ARP cache is poisoned with the attacker's MAC mapped to 10.1.1.1, so all traffic meant for the legitimate host now goes to the attacker instead, who can read it and optionally forward it on (man-in-the-middle) to stay undetected.

Why ARP is exploitable: ARP is stateless, which means a host accepts an ARP reply even if it never sent a matching request, and does not verify that the reply really came from the claimed owner of the IP address.

Real-world use: An attacker on a coffee shop's wired network runs an ARP spoofing attack between a customer's laptop and the router. The customer's web browsing, including any unencrypted logins, now passes through the attacker's machine first.

Tools and commands used here: arpspoof, Ettercap, bettercap (section 3.14).

AI angle: An attacker could ask an AI assistant to explain how to script an ARP spoofing attack responsibly in a lab for learning purposes. A defender could ask: "Write a short explanation for end users describing what ARP spoofing looks like on their machine (slow network, certificate warnings)."

Countermeasures and defense: Dynamic ARP Inspection (section 3.15) using the DHCP snooping binding table.

EXAM FOCUS:

- Keyword "ARP is stateless", "forged ARP request/reply", "poisoned ARP cache" -> ARP spoofing/poisoning.
- Trap: ARP spoofing targets the IP-to-MAC mapping on hosts and switches (Layer 2/3 boundary); it is different from MAC spoofing (section 3.18), which claims someone else's actual MAC address outright rather than lying about which IP maps to which MAC.
- ARP spoofing and ARP poisoning are the same attack, just two names used interchangeably in CEH material.

Memory trick: "ARP never checks ID, so whoever shouts an answer first gets believed. That is the whole vulnerability."

3.14 ARP Spoofing and Poisoning Tools [page 8]

What it is (simple words): Several named tools automate ARP spoofing.

Tools from the slide, with their stated purpose:

- **arpspoof** (linux.die.net): redirects packets from a target host (or all hosts) on the LAN intended for another host, by forging ARP replies.

- **Habu** (github.com): a hacking toolkit with commands for ARP poisoning, sniffing, DHCP starvation, and more.
- **bettercap** (bettercap.org): a full-featured network attack and monitoring framework, includes ARP spoofing modules.
- **Ettercap** (ettercap-project.org): a comprehensive suite for man-in-the-middle attacks, includes ARP poisoning.
- **RITM** (github.com): an ARP spoofing / man-in-the-middle helper tool.
- **ARP Spoofer** (github.com): a dedicated ARP spoofing tool.
- **[UNCLEAR tool name on page 8, reads like "larp"]** (github.com): likely another ARP-related tool; verify the exact name against official courseware.

How it works (numbered steps, generic to this tool family):

1. Pick a target IP (the victim) and, often, a second IP to impersonate (commonly the default gateway).
2. The tool crafts ARP reply packets claiming the attacker's MAC address owns the gateway's IP (and optionally the reverse, claiming to own the victim's IP when talking to the gateway).
3. The tool sends these forged replies repeatedly so they keep overwriting the victim's (and gateway's) ARP cache entries.
4. The attacker's machine, now the man-in-the-middle, must forward traffic between the real victim and real gateway (often with IP forwarding enabled) so the connection keeps working while being sniffed.

What the slide shows: A terminal screenshot of **arpspoof** shows repeated lines of the form "reply [target IP] is-at [attacker MAC]" [UNCLEAR on page 8: exact IP and MAC values shown]. A second terminal screenshot shows **Habu** running an ARP-related command with a password prompt visible [UNCLEAR on page 8: exact command and output].

Real-world use: A tester uses `arpspoof -i eth0 -t <victim-ip> <gateway-ip>` (ADDED CONTEXT syntax, verify for installed version) during an authorized internal penetration test, paired with Wireshark, to prove that credentials for an internal web application are sent in cleartext.

Tools and commands used here: All six tools in the table above.

AI angle: None directly shown; ADDED CONTEXT - an AI assistant can help interpret bettercap or Ettercap log output, which can be dense.

Countermeasures and defense: Dynamic ARP Inspection (next section), static ARP entries for critical hosts, and the general anti-sniffing countermeasures in section 3.33.

EXAM FOCUS:

- Tool-to-purpose recall: "redirects packets by forging ARP replies" -> arpspoof. "Toolkit with ARP poisoning, sniffing, DHCP starvation commands" -> Habu. "Full MITM framework" -> Ettercap or bettercap.
- Trap: Ettercap and bettercap are broader frameworks (support many attacks beyond ARP spoofing); arpspoof is a narrow, single-purpose tool.

Memory trick: "arpspoof lies about one address. Ettercap and bettercap are the whole lying toolbox."

3.15 How to Defend Against ARP Poisoning [page 8]

What it is (simple words): Implement Dynamic ARP Inspection (DAI) using the DHCP snooping binding table. DAI checks the MAC and IP fields of incoming ARP packets against the trusted binding table built by DHCP snooping; if the ARP packet's claim does not match a known good binding, the traffic is blocked.

Analogy: A guest list at a gated community (the DHCP snooping binding table) that records which name (MAC) belongs to which address (IP). Any visitor who claims a different name for that address at the gate (a forged ARP reply) is turned away.

How it works (numbered steps, from the diagram):

1. DHCP snooping is enabled and has already built a binding table that records each legitimate host's MAC address, IP address, lease, VLAN, and interface (example from the slide: `sh ip dhcp snooping binding` output listing a MAC/IP pair with a lease value, type `dhcp-snooping`, and interface [UNCLEAR on page 8: exact hex MAC digits in the sample row]).
2. Dynamic ARP Inspection is enabled on the relevant VLAN.
3. When any ARP packet arrives, the switch checks the MAC and IP fields in it against the binding table entry for the interface it arrived on.
4. If there is no matching entry, or the entry does not match, the switch discards the packet instead of updating any ARP cache with it.
5. This means an attacker's forged ARP reply, claiming to be another host's IP with the attacker's own MAC, is simply dropped at the switch, never even reaching the victim.

Real-world use: A victim's switch port has DAI enabled. When an attacker plugged into another port sends a forged ARP reply claiming to be the gateway, the switch compares the claim to the DHCP snooping binding table, finds no match, and discards it before it ever reaches the victim.

Tools and commands used here: `sh ip dhcp snooping binding` (viewing the binding table that DAI relies on). Full configuration commands are in section 3.16.

AI angle: None directly shown.

Countermeasures and defense: This slide is itself the countermeasure for ARP spoofing (section 3.13).

EXAM FOCUS:

- Keyword "Dynamic ARP Inspection", "DHCP snooping binding table" -> this defense.
- Trap: DAI depends on DHCP snooping already running and having built an accurate binding table; if DHCP snooping is not configured first, DAI has nothing reliable to check against.
- Look-alike: DAI checks ARP packets against IP-MAC bindings; IP Source Guard (section 3.22) does something very similar but for plain IP traffic, not specifically ARP packets.

Memory trick: "DAI = ARP's bouncer, checking every name tag against the DHCP snooping guest list."

3.16 Configuring DHCP Snooping and Dynamic ARP Inspection on Cisco Switches [page 9]

What it is (simple words): This slide walks through the actual Cisco IOS commands and `show` command output used to turn on DHCP snooping and Dynamic ARP Inspection (DAI), and to confirm they are working.

Analogy: This is the step-by-step instruction sheet for building the "guest list" (binding table) and the "ARP bouncer" (DAI) described in sections 3.12 and 3.15.

How it works (numbered command blocks from the slide):

Step 1 - turn on and verify DHCP snooping:

```
Switch(config)# ip dhcp snooping
Switch(config)# ip dhcp snooping vlan 10
Switch# show ip dhcp snooping
```

The `show ip dhcp snooping` output on the slide confirms: DHCP snooping is enabled; DHCP snooping is operational on the following VLANs: 10; DHCP snooping is configured on the following L3 interfaces (a list of trusted interfaces follows, with columns Interface, Trusted, Rate limit).

Step 2 - verify the binding table:

```
Switch# show ip dhcp snooping binding
```

Sample output columns: MacAddress, IpAddress, Lease, Type, VLAN, Interface. The slide's example row shows a MAC address bound to IP 10.10.10.6, a lease value, type `dhcp-snooping`, VLAN 4 [UNCLEAR on page 9: exact hex MAC digits], on interface FastEthernet0/3, with "Total number of bindings: 1" at the bottom.

Step 3 - turn on Dynamic ARP Inspection and verify it:

```
Switch(config)# ip arp inspection vlan 10
Switch# show ip arp inspection
```

The `show ip arp inspection` output on the slide reports: Source Mac Validation: Disabled; Destination Mac Validation: Disabled; IP Address Validation: Disabled; then per-VLAN tables for Configuration/Operation/ACL Match/Static ACL, ACL Logging/DHCP Logging/ Probe Logging, Forwarded/ Dropped/DHCP Drops/ACL Drops counters, DHCP Permits/ ACL Permits/Probe Permits/Source MAC Failures counters, and Dest MAC Failures/IP Validation Failures/Invalid Protocol Data counters, all reading 0 or "Disabled" in this fresh example because no violation has happened yet.

Step 4 - see a DAI violation logged:

```
%SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on
  Fa0/3, vlan 10
```

[UNCLEAR on page 9: the exact IP addresses, MAC address, and timestamp in the full syslog line are hard to read precisely; the message format and the fact that it names the interface and VLAN of the denied ARP packet is the important exam point, not the exact sample values.]

Real-world use: A network engineer follows these exact four steps during a change window: enable snooping, confirm the binding table is populating as real devices renew their leases, enable DAI, and then watch the syslog for `SW_DAI` deny messages to confirm the defense is actually catching spoofed ARP attempts during a planned test.

Tools and commands used here: `ip dhcp snooping`, `ip dhcp snooping vlan <id>`, `show ip dhcp snooping`, `show ip dhcp snooping binding`, `ip arp inspection vlan <id>`, `show ip arp inspection`.

AI angle: None directly shown; ADDED CONTEXT - an AI assistant can help a junior engineer interpret a wall of `show ip arp inspection` counters quickly during an exam-prep review.

Countermeasures and defense: This slide is the configuration detail behind sections 3.12 and 3.15.

EXAM FOCUS:

- Command recall is heavily tested here: know that `show ip dhcp snooping binding` lists the trusted MAC-IP-VLAN-interface data, and that `show ip arp inspection` lists DAI's validation settings and counters.
- Keyword "%SW_DAI-4-DHCP_SNOOPING_DENY" -> a logged, blocked ARP spoofing attempt.
- Trap: enabling `ip arp inspection vlan <id>` without first enabling and populating DHCP snooping for that VLAN leaves DAI with an empty binding table, which can block all ARP traffic including legitimate hosts.

Memory trick: "Snoop first, inspect second, read the log last." The order of the four steps is the order you must configure them in.

3.17 ARP Spoofing Detection Tools [page 9]

What it is (simple words): Beyond building Cisco-level defenses, dedicated tools can actively detect ARP poisoning and ARP flooding attacks and help locate the source of the attack.

Tool highlighted on the slide: Capsa Portable Network Analyzer - it helps security professionals quickly detect ARP poisoning and ARP flooding attacks and locate the attack source (the slide shows its capture window with a live traffic view).

Other ARP spoofing detection tools listed:

Tool	Link
Wireshark	wireshark.org
OpUtils	manageengine.com
netspionage	github.com
NetProbe	github.com
ARP-GUARD	arp-guard.com

How it works (numbered steps, general approach of these tools):

1. The tool continuously monitors ARP traffic on the network.
2. It builds a baseline of which MAC address should be claiming which IP address.
3. It alerts when it sees a MAC-to-IP claim that conflicts with the baseline (a classic sign of ARP spoofing) or sees an abnormal volume of ARP traffic (a classic sign of MAC flooding or an ARP-based scan).
4. Some tools, like Capsa, also try to pinpoint the physical source (by switch port or MAC) of the suspicious traffic.

Real-world use: A help desk ticket reports intermittent connection drops. Running Capsa reveals repeated conflicting ARP replies claiming the gateway's IP from two different MAC addresses, and the security team traces the second MAC to a rogue device under a desk.

Tools and commands used here: Capsa Portable Network Analyzer, Wireshark, OpUtils, netspionage, NetProbe, ARP-GUARD.

AI angle: None directly shown.

Countermeasures and defense: These tools complement, but do not replace, switch-level defenses (DAI, port security); they are mainly for detection and investigation rather than prevention.

EXAM FOCUS:

- Keyword "detects ARP poisoning and flooding, locates the attack source" -> Capsa Portable Network Analyzer.
- Trap: Wireshark can show ARP traffic manually, but it does not automatically flag ARP spoofing the way a dedicated detection tool does; the exam may ask which tool is purpose-built for this versus which is a general packet capture tool.

Memory trick: "Capsa catches the culprit; Wireshark just shows you the wire."

3.18 MAC Spoofing and Duplicating [page 10]

What it is (simple words): A MAC duplicating attack is launched by sniffing a network for MAC addresses of clients who are actively associated with a switch port, and then re-using one of those addresses. By listening to the traffic on the network, a malicious user can intercept and use a legitimate user's MAC address to receive all the traffic destined for that user. This attack allows an attacker to gain access to the network and take over someone's identity on the network.

Analogy: Overhearing a hotel guest's room key code, then using a duplicate key to walk into their room and collect their mail, while they are still a guest at the same hotel.

How it works (numbered steps, from the diagram):

1. A switch port has a rule such as: "Allow access to the network only if your MAC address is xx:xx:xx:xx:xx:ff."
2. The legitimate user's device, with that exact MAC address, is normally the only one permitted on that port.
3. The attacker sniffs the network and discovers that MAC address being used by the legitimate, currently associated user.
4. The attacker changes their own NIC's reported MAC address to match (spoofs it), effectively saying "No, my MAC address is also xx:xx:xx:xx:xx:ff," and connects.

5. The switch now treats the attacker as if they were the legitimate user, delivering traffic meant for that identity (or allowing the attacker onto a port or network that filters by MAC).

> **REMEMBER:** This same technique can be used to bypass a wireless access point's MAC address filtering, since Wi-Fi MAC filtering checks the exact same kind of address that can be spoofed on a wired NIC.

Real-world use: A corporate guest Wi-Fi only allows pre-registered MAC addresses. An attacker sniffs the air for an already-connected guest's MAC address, spoofs it on their own laptop, and connects freely once that guest briefly disconnects.

Tools and commands used here: MAC address change tools are OS-native (for example `ifconfig` or `macchanger` on Linux, Device Manager on Windows, ADDED CONTEXT, verify exact commands for your OS).

AI angle: None directly shown.

Countermeasures and defense: See section 3.22 (DHCP snooping binding table, Dynamic ARP Inspection, and IP Source Guard).

EXAM FOCUS:

- Keyword "re-using another client's MAC address", "take over someone's identity on the network" -> MAC spoofing/duplicating.
- Trap: MAC spoofing claims to be a specific, real, already-known-good device. ARP spoofing (section 3.13) instead lies about which IP maps to which MAC, without necessarily claiming to be a specific other real device's MAC on the wire the same way.
- Keyword "bypasses MAC filtering on a wireless access point" -> this same attack, applied to Wi-Fi.

Memory trick: "MAC spoofing = identity theft. ARP spoofing = lying about an address book entry."

3.19 IRDP Spoofing [page 10]

What it is (simple words): ICMP Router Discovery Protocol (IRDP) is a routing protocol that allows a host to discover the IP addresses of active routers on their subnet, by listening to router advertisement and soliciting messages on their network. The attacker sends a spoofed IRDP router advertisement message to the host on the subnet, causing it to change its default router to whatever the attacker chooses. This attack allows the attacker to sniff the traffic and collect valuable information from the packets. Attackers can use IRDP spoofing to launch man-in-the-middle, denial-of-service, and passive sniffing attacks.

Analogy: A fake road sign dropped on a street corner that tells every driver "this way to the highway", when it actually redirects all traffic through the attacker's driveway, where they can copy down everyone's license plates before letting them continue on their way.

How it works (numbered steps, from the diagram):

1. Normally, a host's routing table points to the legitimate router, which forwards traffic to the internet.
2. The attacker's own router (or a spoofed message) sends a forged IRDP router advertisement onto the subnet.
3. The victim host, trusting the advertisement, updates its routing table to use the attacker's chosen router as its new default gateway.
4. All of the victim's outbound traffic now flows through the attacker's router first, letting the attacker read it (passive sniffing), silently drop it (denial-of-service), or alter and forward it (man-in-the-middle).

Real-world use: On an older network segment that still has IRDP enabled, an attacker injects a forged advertisement and quietly becomes the default gateway for several hosts, harvesting their traffic for a few hours before anyone notices slower routing.

Tools and commands used here: None named specifically on this slide (ADDED CONTEXT, verify: `yersinia` and custom raw-socket scripts have been used historically to craft spoofed ICMP router discovery messages).

AI angle: None directly shown.

Countermeasures and defense: Disable IRDP on hosts and routers where it is not needed (ADDED CONTEXT, since IRDP is largely obsolete and rarely required on modern networks), and prefer statically configured or authenticated routing/gateway information.

EXAM FOCUS:

- Keyword "ICMP Router Discovery Protocol", "host discovers routers by listening to advertisements" -> IRDP.
- Keyword "attacker changes the victim's default router" -> IRDP spoofing.
- Trap: this is similar in *effect* to a rogue DHCP server changing the default gateway (section 3.10), but the *mechanism* is different: IRDP spoofing abuses router advertisement/solicitation messages directly, while a rogue DHCP server abuses the DHCP lease process.

Memory trick: "IRDP = I Redirect your Default Path."

3.20 VLAN Hopping [page 11]

What it is (simple words): VLAN Hopping is a network attack method used to gain unauthorized access to resources on a virtual local area network (VLAN). This type of attack allows an attacker to bypass network segmentation controls, which are put in place to isolate network traffic for security and management reasons. The slide shows two techniques: switch spoofing and double tagging.

Analogy: VLANs are like separate hallways in an office, each needing a different badge to enter. VLAN hopping is sneaking through a service corridor (switch spoofing) or stapling two different visitor badges together so a scanner only reads the outer one and lets you walk into a hallway your real badge never allowed (double tagging).

How it works, Switch Spoofing (numbered steps):

1. Attackers connect a rogue switch onto the network.
2. The rogue switch tricks a legitimate switch into believing it is another legitimate switch that needs a trunk link (trunk links normally carry traffic for multiple VLANs between switches).
3. This creates an unauthorized trunk link between the rogue switch and the legitimate switch.
4. Once trunked, the attacker's rogue switch can see traffic from every VLAN that crosses that trunk, for example VLAN 10 (Server1) and VLAN 20 (Server2) in the diagram.

How it works, Double Tagging (numbered steps):

1. Attackers add and modify tags in the Ethernet frame: they craft a frame with two stacked 802.1Q VLAN tags, an outer tag and an inner tag.
2. The frame enters Switch 1, which strips off only the outer tag (which the attacker deliberately set to match Switch 1's own native VLAN, so it is processed and removed without suspicion) and forwards the frame across the trunk.
3. The inner tag, which names the attacker's real target VLAN, is still on the frame when it reaches Switch 2.
4. Switch 2 now reads this previously hidden inner tag and delivers the frame into the target VLAN (for example VLAN 10 or VLAN 20 in the diagram), even though the attacker's own access port was only ever a member of VLAN 1.
5. This technique allows the flow of traffic through any VLAN in the network, bypassing the segmentation that VLANs are meant to enforce.

Real-world use: An attacker plugged into a VLAN 1 access port crafts double-tagged frames and successfully reaches a finance server that lives on a supposedly isolated VLAN 10, something that should have been impossible from a VLAN 1 port.

Tools and commands used here: Custom frame-crafting tools or scripts (ADDED CONTEXT, Scapy or Yersinia can build double-tagged frames; verify exact syntax).

AI angle: None directly shown.

Countermeasures and defense: See section 3.21.

EXAM FOCUS:

- Keyword "rogue switch tricks a legitimate switch into forming a trunk" -> switch spoofing.
- Keyword "two VLAN tags stacked, outer tag stripped" -> double tagging.
- Trap: double tagging only works one-way (into the target VLAN), and it depends on the attacker's native VLAN matching the trunk's native VLAN; it does not let the attacker receive a reply back the same way.
- Numbers to remember: VLAN 10 and VLAN 20 in the slide's example are just sample VLAN IDs, not fixed values you must memorize.

Memory trick: "Switch spoofing fakes being a switch. Double tagging fakes being two frames in one."

3.21 STP Attack [page 11]

What it is (simple words): Attackers connect a rogue switch into the network to change the operations of the Spanning Tree Protocol (STP) and sniff all the network traffic. Attackers configure the rogue switch so that its priority is lower than that of any other switch in the network, which makes it the root bridge, thus allowing the attacker to sniff all the traffic flowing in the network.

Analogy: STP elects one switch as the "root" of the network tree, like electing a town's central post office that all mail must pass through at some point for sorting. An attacker who rigs the election (by claiming the lowest possible priority number) gets to become that central post office, so all mail flows through them.

How it works (numbered steps, from the diagram):

1. Normally, Switch 1 (priority 32769) and Switch 2 (priority 32769) sit between the real root bridge, the server, and the user, each forwarding traffic along the shortest path that STP has calculated.
2. The attacker connects a rogue switch and sets its bridge priority to 0 (STP elects the switch with the lowest priority number as the root bridge, so priority 0 beats the default 32768/32769 used by the other switches).
3. STP recalculates the topology and the rogue switch becomes the new root bridge.
4. Traffic flow is rerouted so that it now passes through the rogue switch on its way between the server and the user.
5. The attacker, sitting at the root bridge, now sniffs all the network traffic that flows through it.

Real-world use: An attacker plugs a cheap unmanaged-looking switch with its priority manually set to 0 into an open wall jack. Within seconds the real network's spanning tree recalculates, traffic reroutes through the rogue device, and the attacker begins capturing everything.

Tools and commands used here: Yersinia (its STP attack modules, ADDED CONTEXT, same multi-protocol tool named in section 3.11 for DHCP starvation).

AI angle: None directly shown.

Countermeasures and defense: See section 3.24 (BPDU Guard, Root Guard, Loop Guard, UDLD).

EXAM FOCUS:

- Keyword "rogue switch with the lowest bridge priority becomes root bridge" -> STP attack.
- Number to remember: STP elects the lowest priority number as root; the slide's attacker sets priority 0, beating the normal default of 32768 or 32769 used by legitimate switches on the example.
- Trap: this attack needs a rogue switch (a Layer 2 device running STP), not just a rogue host; a single PC cannot become a root bridge by itself without acting as or controlling a switch.

Memory trick: "Lowest number wins the STP election. Priority 0 is an unbeatable, rigged vote."

3.22 How to Defend Against MAC Spoofing [page 12]

What it is (simple words): Use the DHCP Snooping Binding Table, Dynamic ARP Inspection, and IP Source Guard together to defend against MAC spoofing. The switch checks the MAC and IP fields of incoming traffic against the binding table; if the traffic from that interface does not match a known entry, the traffic is blocked.

Analogy: A triple-check at the door: the guest list (DHCP snooping binding table) records who should be where, a second guard (DAI) checks every ARP claim against that list, and a third guard (IP Source Guard) checks every actual packet's source IP and MAC against the same list before letting any traffic through.

How it works (numbered steps, from the diagram):

1. DHCP snooping has already built a binding table, shown on the slide via `sh ip dhcp snooping binding`, with a sample entry: MAC address 2a:33:4c:2f:4a:1c, IP address 10.10.10.9, lease 185235, type dhcp-snooping, VLAN 4, interface FastEthernet3/18.
2. Dynamic ARP Inspection and IP Source Guard are both enabled on the switch.

3. Three hosts are shown: 10.10.10.1 (MAC A, legitimate), 10.10.10.2 (MAC B, legitimate), and 10.10.10.5 (MAC C, the attacker).
4. The attacker (MAC C) sends traffic claiming to be IP 10.10.10.2, which the switch's binding table has already recorded as belonging to MAC B, not MAC C.
5. The switch checks the MAC and IP fields on the interface; since this traffic's claimed IP/MAC pair does not match the binding table entry for that interface, the switch discards the packet. If the entry does match, the packet is forwarded normally.

Real-world use: A switch with IP Source Guard enabled silently drops every packet from a rogue laptop that tries to claim a coworker's IP address, protecting that coworker's session from being hijacked.

Tools and commands used here: `sh ip dhcp snooping binding` plus the IP Source Guard and DAI configuration from sections 3.15 and 3.16.

> **ADDED CONTEXT:** IP Source Guard is enabled per interface with `ip verify source` (optionally `ip verify source port-security` to also check the MAC address, not just the IP). It relies on the same DHCP snooping binding table as DAI (verify exact syntax for your IOS version).

AI angle: None directly shown.

Countermeasures and defense: This slide is the countermeasure itself for MAC spoofing (section 3.18).

EXAM FOCUS:

- Keyword "check MAC and IP fields against the binding table; discard if no match" -> IP Source Guard working together with DHCP snooping and DAI.
- Trap: the exam may test whether you know these three features (DHCP snooping, DAI, IP Source Guard) are meant to work as a set rather than stand-alone; DHCP snooping builds the data the other two depend on.

Memory trick: "Three guards, one guest list: snooping writes it, DAI checks ARP against it, IP Source Guard checks the actual packets against it."

3.23 How to Defend Against VLAN Hopping [page 12]

What it is (simple words): Separate defenses exist for the two VLAN hopping methods: switch spoofing and double tagging.

Defend against Switch Spoofing (numbered steps):

1. Explicitly configure end-user-facing ports as access ports, and ensure they are configured to not negotiate trunks: `switchport mode access`.

2. Ensure that all trunk ports are configured to not negotiate trunks either: `switchport mode trunk` together with `switchport nonegotiate`.

Defend against Double Tagging (numbered steps):

1. Specify the default VLAN explicitly, which is used if the interface stops trunking: `switchport access vlan 2`.
2. Ensure the native VLAN on all trunk ports is changed to an unused VLAN ID, so attackers cannot easily predict or match it: `switchport trunk native vlan 999`.
3. Ensure the native VLAN on all trunk ports is explicitly tagged (rather than left untagged, which is what double tagging relies on): `vlan dot1q tag native`.

Analogy: For switch spoofing: nail every regular office door shut so it can never accidentally become a service corridor door (no trunk negotiation on access ports). For double tagging: give the "default hallway" its own clearly marked, unused badge color (an unused native VLAN ID) and demand every badge be explicitly checked and tagged, so no unmarked badge silently passes through.

Real-world use: A network team audits every switch and finds several access ports still set to `switchport mode dynamic auto` (which can negotiate a trunk if asked). After switching them all to a locked-down `switchport mode access` with `switchport nonegotiate` and moving the native VLAN on every trunk from VLAN 1 to an unused VLAN 999, VLAN hopping attempts in a later penetration test fail completely.

Tools and commands used here: `switchport mode access`, `switchport mode trunk`, `switchport nonegotiate`, `switchport access vlan 2`, `switchport trunk native vlan 999`, `vlan dot1q tag native`.

AI angle: None directly shown.

Countermeasures and defense: This slide is itself the countermeasure for VLAN hopping (section 3.20).

EXAM FOCUS:

- Command recall: match each command to its purpose. `switchport nonegotiate` -> stops automatic trunk negotiation (fixes switch spoofing). `switchport trunk native vlan 999` -> moves the native VLAN off the default (fixes double tagging). `vlan dot1q tag native` -> forces tagging of the native VLAN (also fixes double tagging).
- Trap: VLAN 1 is the default native VLAN on many switches out of the box; leaving it as the native VLAN on trunks is exactly what double tagging exploits, so changing it is the key fix.

Memory trick: "Lock the door (nonegotiate), move the default hallway (native VLAN 999), and tag every badge (dot1q tag native)."

3.24 How to Defend Against STP Attacks [page 13]

What it is (simple words): To prevent an STP attack, four Cisco IOS security features must be implemented together: BPDU Guard, Root Guard, Loop Guard, and UDLD (Unidirectional Link Detection).

Analogy: Four separate locks on the election process for "which switch becomes the town's central post office" (the root bridge): one lock refuses to even listen to election ballots from untrusted doors (BPDU Guard), one refuses to accept a new winner from a door that should never produce one (Root Guard), one stops a ballot loop from confusing the count (Loop Guard), and one checks that both ends of a wire agree who is on the other end at all (UDLD).

How it works, BPDU Guard (to enable on all PortFast edge ports):

```
configure terminal
interface gigabitethernet slot/port
spanning-tree portfast bpduguard
```

BPDU Guard disables a port the moment it receives a Bridge Protocol Data Unit (BPDU, the STP control message), since a true end-user edge port should never see BPDUs from a legitimate switch in the first place; this stops an attacker's rogue switch from being able to participate in STP through that port at all.

How it works, Root Guard (to enable the root guard feature on an interface):

```
configure terminal
interface gigabitethernet slot/port
spanning-tree guard root
```

Root Guard stops a port from ever becoming the path to a new root bridge; if a BPDU arrives on that port claiming a better (lower) priority than the current root, the port is put into a blocking state instead of honoring it. This directly defeats an attacker claiming priority 0.

How it works, Loop Guard:

```
configure terminal
interface gigabitethernet slot/port
spanning-tree portfast bpduguard
```

[UNCLEAR on page 13: the slide shows this same command text under the Loop Guard heading as it does under BPDU Guard, which may be a slide error or duplication. ADDED CONTEXT, verify: the standard Cisco IOS command for Loop Guard is normally

`spanning-tree guard loop` (applied per interface), which prevents a port from incorrectly transitioning to a forwarding state when it stops receiving BPDUs, a condition that can create a bridging loop.]

How it works, UDLD (to enable UDLD on an interface):

```
configure terminal
interface gigabitethernet slot/port
udld {enable | disable | aggressive}
```

UDLD detects unidirectional links (a link that can send but not receive, or vice versa, often from a wiring or transceiver fault, but also relevant if a rogue device interferes with only one direction of a link) and can shut the port down to prevent the kind of inconsistent state that STP attacks can also exploit.

Real-world use: A security-hardened switch configuration applies BPDU Guard and Root Guard to every access port, Loop Guard to redundant non-edge links, and UDLD aggressively on fiber trunk links. An attacker's rogue switch plugged into any access port is immediately shut down by BPDU Guard before it can even attempt to influence the spanning tree.

Tools and commands used here: `spanning-tree portfast bpduguard`, `spanning-tree guard root`, (Loop Guard command, see UNCLEAR note above), `udld enable` / `udld disable` / `udld aggressive`.

AI angle: None directly shown.

Countermeasures and defense: This slide is itself the countermeasure for the STP attack (section 3.21).

EXAM FOCUS:

- Match each STP defense to its exact job: BPDU Guard disables ports that should never see BPDUs at all (edge/access ports). Root Guard stops a specific port from ever becoming the path to a new root bridge. Loop Guard prevents a port from wrongly entering a forwarding state and creating a loop. UDLD catches links that are only working in one direction.
- Trap: BPDU Guard and Root Guard sound similar but differ: BPDU Guard is usually for edge ports where no BPDU should ever appear at all; Root Guard is for non-edge ports where BPDUs are expected but a *root claim* from that direction should never be honored.

Memory trick: "B-R-L-U: BPDU Guard bars BPDUs entirely, Root Guard Rejects root claims, Loop Guard Locks out false-forwarding, UDLD Uncovers dead-end directions."

3.25 DNS Poisoning Techniques [page 13]

What it is (simple words): DNS poisoning is a technique that tricks a DNS server into believing it has received authentic information when it has not. It results in the substitution of a false IP address at the DNS level, where web addresses are converted into numeric IP addresses. It allows the attacker to replace the IP address entries for a target site on a given DNS server with the IP address of a server he or she controls. Attackers use DNS poisoning tools [UNCLEAR on page 13: tool name reads similarly to "DerpNSpoof"; verify the exact product name against official courseware] to create fake DNS entries for the server, containing malicious content, with names similar to that of the target server.

Analogy: DNS is the phone book that turns a business name into a phone number. DNS poisoning is secretly editing entries in that phone book so that dialing a trusted business's listed number actually rings the attacker's phone instead.

How it works (overview of the four types, each covered in detail next):

1. **Intranet DNS Spoofing:** works on the local area network (LAN), usually paired with ARP poisoning.
2. **Internet DNS Spoofing:** works across the wider internet (remote network), usually by changing the victim's configured DNS server via a Trojan.
3. **Proxy Server DNS Poisoning:** changes the victim's browser proxy setting via a Trojan, so all web traffic is funneled through the attacker before resolving anything.
4. **DNS Cache Poisoning:** forges and inserts false records directly into a DNS resolver's cache, affecting everyone who later queries that resolver.

Real-world use: A victim types their bank's real web address, but because the DNS answer they received was poisoned, their browser connects to a server that only looks like the bank, letting the attacker harvest login credentials before quietly forwarding the connection on.

Tools and commands used here: The DNS poisoning tool noted on the slide [UNCLEAR, see above]; specific tools and Trojans named in sections 3.26 to 3.28.

AI angle: None directly shown; ADDED CONTEXT - an attacker could use an AI assistant to generate convincing fake login pages to host on the poisoned domain.

Countermeasures and defense: See section 3.30 (14-point DNS spoofing defense list).

EXAM FOCUS:

- Keyword "tricks a DNS server into accepting false information" -> DNS poisoning/spoofing in general.

- Must-memorize list: the four types (Intranet, Internet, Proxy Server, Cache Poisoning). The exam commonly gives a scenario and asks which of the four fits.
- Trap: all four types end with the victim reaching a fake site, but the *mechanism* that gets them there differs; read the scenario carefully for whether it is LAN-only (intranet), a changed DNS server (internet), a changed proxy (proxy server), or a poisoned shared cache (cache poisoning).

Memory trick: "I-I-P-C: Intranet, Internet, Proxy, Cache" for the four DNS poisoning types, in the order the module explains them.

3.26 Intranet DNS Spoofing [page 14]

What it is (simple words): In this technique, the attacker's system must be connected to the local area network (LAN) and be able to sniff packets. It works well against switches, combined with ARP Poison Routing.

Analogy: Rewiring the office's own internal directory board so that "accounting department" now points down the hallway to the attacker's office instead of the real one, but this trick only works because the attacker is physically inside the same building (the LAN).

How it works (numbered steps, from the diagram):

1. John (IP 10.0.0.3) asks, "What is the IP address of www.example.com?" His browser sends a DNS request.
2. The DNS request is meant to travel to the router (10.0.0.254).
3. Because the attacker has already poisoned the router using an ARP spoofing/ARP poison routing technique (see section 3.13), the attacker redirects the DNS request to the attacker's own machine instead.
4. The attacker has set up a fake website in advance, and sends a DNS response claiming that www.example.com is located at the attacker's own IP (10.0.0.5 in the slide's example).
5. John's browser, trusting the response, connects to 10.0.0.5, the fake website, instead of the real website (reached via the real public IP, 200.0.0.45 in the slide's example).
6. The attacker's fake website sniffs the credentials John enters and then redirects the request on to the real website, so John notices nothing wrong.

Real-world use: An attacker inside a coffee shop's Wi-Fi network combines ARP poisoning with intranet DNS spoofing to intercept login attempts to a popular webmail service, all without ever leaving the local network.

Tools and commands used here: ARP poisoning tools from section 3.14 (arpspoof, Ettercap, bettercap), plus DNS spoofing tools such as `dnsspoof` (part of the dsniff suite, ADDED CONTEXT, same family as macof).

AI angle: None directly shown.

Countermeasures and defense: See section 3.30; specifically, defending against ARP poisoning (section 3.15) also weakens this attack, since it depends on ARP poison routing to work.

EXAM FOCUS:

- Keyword "attacker must be on the LAN", "works with ARP Poison Routing" -> Intranet DNS Spoofing.
- Trap: this is the one DNS poisoning type that absolutely requires local network access; the other three (sections 3.27 to 3.29) can be done remotely via a Trojan or a poisoned shared resolver.

Memory trick: "Intranet = Inside. The attacker must already be in the building."

3.27 Internet DNS Spoofing [page 14]

What it is (simple words): In Internet DNS Spoofing, the attacker infects John's machine with a Trojan and changes his DNS IP address to that of the attacker's own server. Unlike intranet DNS spoofing, the attacker does not need to be on the same LAN.

How it works (numbered steps, from the diagram):

1. The attacker infects John's computer with a Trojan that changes his configured DNS server IP address to the attacker's own, for example 200.0.0.2.
2. John (IP 10.0.0.3) asks, "What is the IP address of www.example.com?" His browser sends the DNS request, but now it goes straight to the attacker's DNS server at 200.0.0.2, because that is what John's machine is configured to use.
3. The attacker runs a malicious DNS server at that same IP (200.0.0.2).
4. The attacker's DNS server sends back a DNS response pointing www.example.com to a fake website IP, 65.0.0.2 in the slide's example.
5. John's browser connects to 65.0.0.2, the fake website, instead of the real website (www.easysecurity.com at 200.0.0.45 in the slide's example).
6. The attacker's fake website sniffs the credential John enters and redirects the request on to the real website.

Analogy: Instead of rewiring the office directory board (intranet version), the attacker secretly swaps out John's personal phone book (his DNS server setting) for one they printed themselves, so every lookup John makes from then on, anywhere in the world, goes through the attacker's fake listings.

Real-world use: A victim downloads a cracked game installer that is secretly a Trojan. From that day forward, every DNS lookup on the victim's machine silently resolves through the attacker's DNS server, letting the attacker redirect banking or email domains at will, with no need to ever be near the victim physically.

Tools and commands used here: A Trojan capable of changing OS-level DNS settings, plus a malicious DNS server (ADDED CONTEXT, tools such as `dnschef` can stand up a rogue DNS server for testing; verify legitimate use only).

AI angle: None directly shown.

Countermeasures and defense: See section 3.30; endpoint protection and anti-malware to prevent or detect the Trojan infection itself is also critical here (see Module 7's malware countermeasures).

EXAM FOCUS:

- Keyword "Trojan changes the DNS server setting", "no LAN access needed" -> Internet DNS Spoofing.
- Trap: Internet DNS Spoofing and Proxy Server DNS Poisoning (next section) both use a Trojan on the victim's machine, but they change different settings: DNS server address versus browser proxy address. Read carefully which setting the scenario describes.

Memory trick: "Internet DNS Spoofing swaps John's whole phone book (DNS server). Proxy poisoning (next) swaps his mail forwarding address instead (proxy)."

3.28 Proxy Server DNS Poisoning [page 15]

What it is (simple words): The attacker sends a Trojan to John's machine that changes his proxy settings (for example, in Internet Explorer) to point to the attacker's own proxy server, and redirects his traffic to a fake website.

How it works (numbered steps, from the diagram):

1. The attacker infects John's computer by changing his Internet Explorer proxy address setting to the attacker's own IP, 200.0.0.2 in the slide's example.
2. John (IP 10.0.0.5) now has all of his web requests pass through the attacker's machine first, because every browser request is routed through whatever proxy address is configured.
3. The attacker runs a proxy server at that IP (200.0.0.2).
4. The attacker's proxy sends John's request on to a fake website (65.0.0.2 in the slide's example) instead of the real site he asked for.
5. The attacker's fake website sniffs John's credential and redirects the request on to the real website (www.easysecurity.com at 200.0.0.45 in the slide's example), so John's session appears to work normally.

Analogy: Convincing John to route all of his outgoing mail through a fake post office box the attacker controls (the proxy setting), instead of tampering with how addresses are looked up at all; the attacker simply reads and resends everything that passes through their box.

Real-world use: A Trojan silently sets a victim's browser proxy to an attacker-controlled server. Every single web request, regardless of what domain the victim types, is funneled through that attacker proxy first, giving the attacker a perfect, centralized vantage point.

Tools and commands used here: A Trojan capable of modifying browser proxy settings, plus a malicious proxy server (ADDED CONTEXT, tools such as `mitmproxy` or `squid` configured maliciously could play this role in a lab; verify legitimate use only).

AI angle: None directly shown.

Countermeasures and defense: See section 3.30; also, endpoint protection to catch the Trojan before it can modify browser settings.

EXAM FOCUS:

- Keyword "changes the proxy settings", "Internet Explorer proxy" -> Proxy Server DNS Poisoning.
- Trap: despite its name including "DNS Poisoning", this attack does not actually poison any DNS record at all; it works entirely by redirecting traffic through a changed proxy setting. The exam may test whether you notice this naming trap.

Memory trick: "Proxy poisoning doesn't touch DNS at all. It just makes John mail everything through the attacker's mailbox first."

3.29 DNS Cache Poisoning [page 15]

What it is (simple words): DNS cache poisoning refers to altering or adding forged records into the DNS resolver's cache, so that a DNS query is redirected to a malicious site. If the DNS resolver cannot validate that the DNS responses it received truly came from an authoritative source, it will cache the incorrect entries locally and serve them to every user who later makes a similar request.

Analogy: Slipping a forged page into a library's shared reference book (the resolver's cache) before anyone checks its source. Every patron who later looks up that entry reads the forged page, not the real one, until the forged page expires or is corrected.

How it works (numbered steps, from the diagram):

1. A user asks, "What is the IP address of www.easysecurity.com?"
2. The query for this DNS info goes to the Internal DNS (resolver).
3. The Internal DNS, not having a cached answer yet, queries the Authoritative DNS server that is supposed to be responsible for easysecurity.com.
4. An attacker (running a rogue DNS server) races to send back a forged DNS response claiming to be from the authoritative source, carrying the IP address of a fake website.

5. Because the Internal DNS cannot validate that this response truly came from the real authoritative server, it accepts and caches the incorrect entry.
6. The user is redirected to the fake website; the attacker's fake website sniffs the credential and redirects the request on to the real website.
7. Every other user who queries that same Internal DNS for easysecurity.com, until the poisoned cache entry expires or is corrected, is served the same forged, malicious answer.

Real-world use: An attacker poisons a popular ISP's resolver cache with a forged record for a banking domain. Thousands of the ISP's customers are silently redirected to a phishing site for as long as the poisoned entry remains cached, making this one of the highest-impact DNS attacks because of its blast radius.

Tools and commands used here: A rogue DNS server capable of racing legitimate authoritative responses (ADDED CONTEXT, tools such as `dnschef` or custom scripts; verify legitimate use only).

AI angle: None directly shown.

Countermeasures and defense: See section 3.30, especially DNSSEC, which is specifically designed to let a resolver cryptographically validate that a DNS response really came from the claimed authoritative source.

EXAM FOCUS:

- Keyword "forged records added to a resolver's cache", "resolver cannot validate the source" -> DNS Cache Poisoning.
- Trap: DNS Cache Poisoning affects every user of the poisoned resolver, not just one targeted victim; this is its biggest difference from the other three DNS poisoning types, which are each aimed at a single victim's machine or LAN.
- Defense to always pair with this attack: DNSSEC.

Memory trick: "Cache poisoning is the only DNS attack in this module that poisons the well for everyone who drinks from it, not just one victim."

3.30 How to Defend Against DNS Spoofing [page 16]

What it is (simple words): A 14-point checklist of countermeasures against all forms of DNS spoofing and poisoning.

The 14 points:

1. Implement a Domain Name System Security Extension (DNSSEC).
2. Use a Secure Socket Layer (SSL) for securing the traffic.
3. Resolve all DNS queries to a local DNS server.
4. Block DNS requests to external servers.
5. Configure a firewall to restrict external DNS lookup.

6. Implement an intrusion detection system (IDS) and deploy it correctly.
7. Configure the DNS resolver to use a new random source port for each outgoing query.
8. Restrict the DNS recursing service, full or partial, to authorized users.
9. Use DNS Non-Existent Domain (NXDOMAIN) rate limiting.
10. Secure internal machines.
11. Use a static ARP and IP tables.
12. Use Secure Shell (SSH) encryption.
13. Do not allow outgoing traffic to use UDP port 53 as a default source port.
14. Audit the DNS server regularly to remove vulnerabilities.

Analogy: A locksmith's full checklist for a vault, not just one lock: verify every delivery's seal (DNSSEC), encrypt the vault's intercom (SSL/SSH), never trust an outside messenger directly (local DNS, block external, firewall), watch for intruders (IDS), vary the guard's patrol route (random source port, NXDOMAIN rate limiting), limit who can ask the vault for directions (restrict recursion), and audit the whole vault regularly.

How it works (how each point maps to an attack): DNSSEC (point 1) directly stops cache poisoning (section 3.29) by letting a resolver cryptographically verify the authoritative source. SSL/SSH (points 2, 12) protect the actual credentials even if a DNS redirect succeeds. Local DNS resolution, blocking external DNS, and firewalling DNS lookups (points 3 to 5) reduce the attack surface for internet DNS spoofing (section 3.27). IDS (point 6) can catch suspicious DNS traffic patterns. Random source ports and NXDOMAIN rate limiting (points 7, 9) make it much harder for an attacker to win the race to forge a matching response. Restricting DNS recursion (point 8) stops attackers from abusing an open resolver. Securing internal machines (point 10) reduces the chance a Trojan ever gets installed in the first place (sections 3.27, 3.28). Static ARP and IP tables (point 11) reduce the risk of the ARP poisoning that intranet DNS spoofing depends on. Blocking UDP/53 as an outgoing source port by default (point 13) closes off a common spoofing trick. Regular audits (point 14) catch misconfigurations before attackers do.

Real-world use: An enterprise rolls out DNSSEC validation on its resolvers, forces all internal clients to use the internal DNS server only, and enables random source ports; a later red team exercise that previously succeeded with cache poisoning now fails outright.

Tools and commands used here: DNSSEC-capable DNS software, firewall rules, IDS, SSH, SSL/TLS.

AI angle: None directly shown; ADDED CONTEXT - an AI assistant can turn this 14-point list into a configuration checklist tailored to a specific DNS server product (BIND, Windows DNS, etc.).

Countermeasures and defense: This slide is itself the full countermeasure set for all of sections 3.25 to 3.29.

EXAM FOCUS:

- Keyword "DNSSEC" -> the single most important, specifically-named defense against DNS cache poisoning; expect it to be the answer whenever a question asks for the best DNS integrity fix.
- Must-memorize: do not use UDP port 53 as the default *source* port for outgoing queries (point 13); random source ports (point 7) make cache-poisoning race attacks far harder to win.
- Trap: SSL/SSH protect data in transit, they do not fix the DNS resolution problem itself; a question about "stopping the DNS redirection" wants DNSSEC or resolver hardening, not encryption.

Memory trick: Group the 14 into four buckets: "Verify" (DNSSEC), "Encrypt" (SSL, SSH), "Restrict" (local DNS, block external, firewall, recursion limits, UDP/53), "Watch" (IDS, NXDOMAIN rate limiting, audits, secure internal machines, static ARP/IP tables).

3.31 Display Filters in Wireshark [page 16]

What it is (simple words): Display filters are used to change the view of packets in a file that has already been captured. They do not change what was captured; they only change what you are currently looking at.

Analogy: A captured file is the full security camera footage for the day. A display filter is choosing to watch only the clips that show one specific person walking by a specific door, without deleting any of the other footage.

How it works (numbered filter categories from the slide):

1. **Display filtering by protocol:** type the protocol name directly in the filter box. Example protocols shown: `arp`, `http`, `tcp`, `udp`, `dns`, or `ip`.
2. **Monitoring specific ports:** `text`

   ```
   tcp.port==23
   ip.addr==192.168.1.100
   ip.addr==192.168.1.100 && tcp.port==23
   ```
3. **Filtering by multiple IP addresses:** `text`

   ```
   ip.addr == 10.0.0.4 or ip.addr == 10.0.0.5
   ```
4. **Filtering by a single IP address:** `text`

   ```
   ip.addr == 10.0.0.4
   ```
5. **Other filters:** `text`

   ```
   ip.dst == 10.0.1.50 && frame.pkt_len > 400
   ip.addr == 10.0.1.12 && icmp && frame.number > 15
     && frame.number < 30
   ip.src==205.153.63.30 or ip.dst==205.153.63.30
   ```

Real-world use: An analyst has a 500 MB capture file from a suspected sniffing incident. Instead of scrolling through everything, they type `tcp.port==23` to instantly isolate every Telnet session, since Telnet is a classic cleartext protocol worth checking first.

Tools and commands used here: Wireshark's display filter bar.

AI angle: An analyst can ask an AI assistant to translate a plain description ("show me only ICMP packets between frame 15 and frame 30 involving 10.0.1.12") into the correct Wireshark filter syntax, exactly as shown in the fourth "Other filters" example above.

Countermeasures and defense: Not applicable; this is an analysis feature, not an attack or a defense by itself, though it is the core skill used to confirm whether sniffing-related attacks succeeded.

EXAM FOCUS:

- Syntax recall: `&&` means AND, `or` (or `||`) means OR, `==` means equals, in Wireshark display filter syntax.
- Keyword "tcp.port==23" -> filtering for Telnet traffic specifically, since 23 is the Telnet port.
- Trap: a display filter (what you see) is different from a capture filter (what Wireshark actually records in the first place); capture filters use a different, BPF-based syntax (ADDED CONTEXT, verify).

Memory trick: "Protocol name alone, or `field == value`, joined with `&&` for AND and `or` for OR."

3.32 Sniffing Tools [page 17]

What it is (simple words): A set of named packet-capture and analysis products used for sniffing, both for attack testing and legitimate network monitoring.

Tools highlighted on the slide:

- **Capsa Portable Network Analyzer** - a portable network performance analysis and diagnostics tool that provides packet capture and analysis capabilities with an easy-to-use interface.
- **OmniPeek** - a sniffer that displays a Google Map in the OmniPeek capture window, showing the locations of all the public IP addresses of captured packets.

Other tools listed:

Tool	Link
RITA (Real Intelligence Threat Analytics)	github.com
Observer Analyzer	viavisolutions.com

Tool	Link
PRTG Network Monitor	paessler.com
Network Performance Monitor	solarwinds.com

Analogy: Capsa is a handheld, easy-to-carry stethoscope for a network. OmniPeek is the same idea but with a world map overlay showing where every captured conversation's other party is physically located.

How it works (general sniffing tool workflow):

1. Install the tool on a machine connected to the segment of interest (or to a SPAN/mirror port, see section 3.5).
2. Start a capture session.
3. Apply filters (display filters, section 3.31) to focus on traffic of interest.
4. Review captured sessions, decode protocols, and export evidence.

Real-world use: A network administrator uses PRTG Network Monitor for day-to-day performance dashboards, and reaches for Capsa only when a deeper, packet-level look is needed to diagnose a specific problem.

Tools and commands used here: Capsa, OmniPeek, RITA, Observer Analyzer, PRTG Network Monitor, Network Performance Monitor.

AI angle: None directly shown; ADDED CONTEXT - RITA in particular is built to analyze network traffic for signs of command-and-control (C2) beaconing, a task that increasingly uses machine learning to spot patterns.

Countermeasures and defense: Not applicable; these are the sniffer tools themselves, which the rest of the module's countermeasures are designed to defeat.

EXAM FOCUS:

- Keyword "Google Map in the capture window showing public IP locations" -> OmniPeek.
- Keyword "portable, easy-to-use, packet capture and diagnostics" -> Capsa Portable Network Analyzer.
- Keyword "Real Intelligence Threat Analytics" -> RITA.
- Trap: PRTG and SolarWinds NPM are broad network *monitoring* platforms (uptime, bandwidth, alerts), not dedicated deep-packet sniffers like Capsa or OmniPeek; a question asking for "a tool that captures and decodes packet contents" should point to Capsa, OmniPeek, or Wireshark, not PRTG or NPM.

Memory trick: "OmniPeek shows you the world map. Capsa fits in your pocket."

3.33 How to Defend Against Sniffing [pages 17 and 18]

What it is (simple words): A 15-point checklist of general countermeasures to defend a network against sniffing attacks of all kinds, split across two slides.

Points 1 to 7 [page 17]:

1. Restrict physical access to the network media, to ensure a packet sniffer cannot be installed.
2. Use end-to-end encryption to protect confidential information.
3. Permanently add the MAC address of the gateway to the ARP cache.
4. Use static IP addresses and ARP tables to prevent attackers from adding spoofed ARP entries for machines in the network.
5. Turn off network identification broadcasts, and if possible, restrict the network to authorized users, to protect it from being discovered with sniffing tools.
6. Use IPv6 instead of the IPv4 protocol.
7. Use encrypted sessions such as SSH instead of Telnet, Secure Copy (SCP) instead of FTP, and SSL for email connections, to protect wireless network users against sniffing attacks.

Points 8 to 15 [page 18, "Cont'd"]:

1. Use HTTPS instead of HTTP to protect usernames and passwords.
2. Use a switch instead of a hub, as a switch delivers data to the intended recipient only.
3. Use Secure File Transfer Protocol (SFTP) instead of FTP for the secure transfer of files.
4. Use PGP and S/MIME, VPN, IPsec, SSL/TLS, Secure Shell (SSH), and One-time passwords (OTPs).
5. Always encrypt wireless traffic with a strong encryption protocol such as WPA2 and WPA3.
6. Retrieve the MAC address directly from the NIC instead of the OS; this prevents MAC address spoofing.
7. Use tools to determine if any NICs are running in promiscuous mode.
8. Use access-control lists (ACLs) to allow access only to a fixed range of trusted IP addresses in a network.

Analogy: This is the full security review checklist for a building after learning every way a wiretap could be installed: lock the wiring closets (physical access), speak in code everywhere (encryption), keep a fixed guest list (static ARP/IP, ACLs), and regularly check every room for hidden listening devices (promiscuous mode detection tools).

How it works (how the points group together):

1. **Reduce physical and broadcast exposure:** points 1, 5, 9.

2. **Encrypt everything, at every layer:** points 2, 6 (note: IPv6 is not itself encryption, but it has native IPsec support more consistently than IPv4), 7, 8, 10, 11, 12.
3. **Lock down addressing:** points 3, 4, 13, 15.
4. **Actively look for sniffers:** point 14 (tied directly into section 3.34's detection techniques).

Real-world use: An organization migrating from Telnet and FTP to SSH and SFTP (points 7 and 10) across all its network devices immediately removes the most commonly sniffed cleartext credentials from its network, even before any of the other 13 points are implemented.

Tools and commands used here: SSH, SCP, SFTP, SSL/TLS, HTTPS, VPN, IPsec, PGP, S/MIME, WPA2/WPA3, ACLs.

AI angle: None directly shown; ADDED CONTEXT - an AI assistant can turn this 15-point list into a prioritized remediation plan for a specific organization's existing protocol usage.

Countermeasures and defense: This slide pair is the general countermeasure set for the whole module.

EXAM FOCUS:

- Must-memorize pairs: Telnet to SSH, FTP to SFTP or SCP, HTTP to HTTPS, WEP to WPA2/WPA3. Every one of these swaps a cleartext protocol for an encrypted one.
- Keyword "retrieve the MAC directly from the NIC instead of the OS" -> prevents MAC address spoofing (point 13), a subtle but testable detail.
- Trap: "use a switch instead of a hub" (point 9) stops *passive* sniffing, but does not by itself stop *active* sniffing techniques like MAC flooding or ARP spoofing covered earlier in this module; those need the dedicated Cisco-level defenses.

Memory trick: "Lock it, encrypt it, fix the addresses, go looking for bugs." Four buckets for all 15 points.

3.34 How to Detect Sniffing [page 18]

What it is (simple words): Three general approaches to find out whether a sniffer is already running somewhere on the network.

The three approaches:

1. **Check the devices running in promiscuous mode:** you need to check which machines are running in promiscuous mode. Promiscuous mode allows a network device to intercept and read each network packet that arrives in its entirety, which is exactly what a sniffer needs to work.

2. **Run IDS:** run an intrusion detection system (IDS) and see if the MAC address of any of the machines has changed (for example, the router's MAC address appearing to change). An IDS can alert the administrator about suspicious activities like this.
3. **Run network tools:** run network tools such as Capsa Portable Network Analyzer to monitor for strange packets. These tools enable you to collect, consolidate, centralize, and analyze traffic data across different network resources and technologies.

Analogy: Checking which rooms have a hidden microphone installed (promiscuous mode check), hiring a guard who notices if anyone's name badge suddenly looks different (IDS watching for MAC changes), and bringing in a specialist with sensitive listening equipment to sweep the whole building (dedicated network tools).

How it works (numbered steps, combining all three):

1. Deploy a tool (section 3.17's list, or OS-level utilities) that can query NICs or probe hosts to reveal which ones are in promiscuous mode.
2. Configure an IDS to baseline normal MAC-to-IP pairings and alert on any unexpected change, since a sniffing or spoofing attack often disturbs this pairing.
3. Run a dedicated network analysis tool regularly to centralize traffic data and spot anomalies that a single host-level check would miss.

Real-world use: A security team schedules a monthly sweep that checks every host for promiscuous mode, reviews IDS alerts for MAC address changes from the past month, and runs Capsa for a full day to catch anything unusual, turning detection into a routine process rather than a one-time reaction.

Tools and commands used here: Capsa Portable Network Analyzer, an IDS platform (ADDED CONTEXT, Snort or Suricata are common examples), and promiscuous-mode detection utilities (see section 3.35 for the specific techniques these utilities use).

AI angle: None directly shown; ADDED CONTEXT - an AI assistant can help write an IDS rule that specifically watches for repeated MAC address changes tied to a single IP, a common ARP spoofing signature.

Countermeasures and defense: Detection complements the 15-point prevention checklist in section 3.33; prevention stops the attack, while detection catches it if prevention already failed or was incomplete.

EXAM FOCUS:

- Keyword "check which machines are in promiscuous mode" -> this is the broad category; section 3.35 gives the exact techniques (ping, DNS, ARP methods) used to do this check.
- Keyword "IDS alerts on MAC address change" -> IDS-based sniffer detection.

- Trap: detecting a sniffer is not the same as stopping one; a question about "how do you find out if a sniffer is already running" wants one of these three detection approaches, not a prevention measure like encryption.

Memory trick: "Check the mode, watch the IDS, bring in the tool."

3.35 Sniffer Detection Techniques: Ping Method and DNS Method [page 19]

What it is (simple words): Two specific, practical techniques to detect whether a particular suspect machine is running a sniffer.

Ping Method (how it works, numbered steps):

1. The investigator sends a ping message to the suspect machine's correct IP address, but deliberately using an incorrect (wrong) MAC address in the Ethernet frame.
2. On a machine *not* running a sniffer (non-promiscuous mode), the Ethernet adapter checks the destination MAC address, sees it does not match its own, and rejects the frame outright; the ping gets no response.
3. On a machine *running* a sniffer (promiscuous mode), the NIC does not reject packets with a different MAC address, since promiscuous mode means it accepts everything; the operating system's network stack still processes the ping at the IP layer and replies.
4. A response received to this deliberately mismatched-MAC ping strongly suggests the target machine's NIC is in promiscuous mode, and is therefore likely running a sniffer.

DNS Method (how it works, numbered steps):

1. Most sniffers perform reverse DNS lookups on the IP addresses found in captured traffic, in order to identify which machine a given IP address belongs to (turning numbers into readable hostnames for the analyst).
2. An investigator sends ping messages to several IP addresses on the network that the suspect should have no reason to be looking up (for example 192.168.0.1, 192.168.0.2, 192.168.0.3 in the slide's example).
3. The investigator then watches for reverse DNS lookup traffic generated in response to these pings.
4. A machine that generates reverse DNS lookup traffic for IP addresses it was never the intended destination of (the ping was not addressed to it) is very likely to be running a sniffer, since it must have captured the ping traffic to even know those IP addresses existed.

Analogy: The Ping Method is handing out an envelope addressed to the wrong apartment number on purpose; only a nosy neighbor who opens everyone's mail (promiscuous mode) will respond. The DNS Method is noticing that someone who was never sent a letter at all somehow still asked the post office, "Who lives at this address?", proving they must have secretly seen mail meant for someone else.

Real-world use: A security analyst suspects one specific desktop of running a rogue sniffer. They ping it using its real IP but a fake MAC address; it responds, confirming promiscuous mode. As a second confirmation, they watch DNS query logs and see that same desktop performing reverse DNS lookups on IP addresses that were never communicating with it directly.

Tools and commands used here: `ping` with a manually spoofed destination MAC address (ADDED CONTEXT: this typically requires a custom tool or raw-socket script, since standard `ping` utilities do not let you set an arbitrary destination MAC directly; verify tool availability), plus DNS query logging or monitoring tools.

AI angle: None directly shown.

Countermeasures and defense: Section 3.33's countermeasures (especially point 14, using tools to determine if any NICs are running in promiscuous mode) connect directly to this detection method.

EXAM FOCUS:

- Keyword "ping with the correct IP but an incorrect MAC address; only a promiscuous NIC responds" -> the Ping Method.
- Keyword "reverse DNS lookup traffic for addresses the host should not know about" -> the DNS Method.
- Trap: the Ping Method only reliably detects a *local* sniffer on the same segment (because it relies on Ethernet-level MAC filtering behavior); it does not detect sniffing happening on a different, unreachable segment.

Memory trick: "Wrong MAC flushes out a nosy NIC (Ping Method). Unwanted DNS lookups flush out a nosy resolver (DNS Method)."

3.36 Sniffer Detection Techniques: ARP Method [page 19]

What it is (simple words): A third sniffer detection technique that relies on how ARP caching behaves differently on a machine running in promiscuous mode compared to every other machine on the segment.

How it works (numbered steps, from the diagram):

1. Only the machine that is in promiscuous mode (labeled "machine C" in the slide's example) caches ARP information (the IP-to-MAC address mapping) for traffic that was never actually addressed to it, simply because promiscuous mode lets it see and process that traffic in the first place.
2. The investigator sends a non-broadcast ping message to a specific host on the network.
3. The machine in promiscuous mode responds to this ping, because it already has the correct cached information about the host that sent the ping request (it learned this earlier, from traffic it was not supposed to be able to see).
4. Every other machine on the network, which never saw that earlier traffic because it was not addressed to them, does not have this information cached and must instead send out its own ARP probe/request first to identify the source of the ping request, before it could even reply.
5. The machine that answers immediately, without needing to ARP-probe for the sender's MAC address first, is exposed as the one running the sniffer.

Analogy: A quiz where only one student in the room already secretly read the test questions ahead of time (because they were eavesdropping on the teacher's private phone call). When asked a question, that student answers instantly, while every honest student has to pause and ask a clarifying question first. The instant, unprompted answer gives the cheater away.

Real-world use: A tester sends a carefully crafted, non-broadcast ping to a decoy host and watches which machine on the segment responds without first needing to ARP-probe for the decoy's MAC address, confirming that machine has been passively capturing traffic it was never meant to see.

Tools and commands used here: Custom ARP/ping probing scripts or dedicated sniffer-detection tools (ADDED CONTEXT, verify availability); this is conceptually similar to what tools like Capsa or ARP-GUARD automate at a larger scale.

AI angle: None directly shown.

Countermeasures and defense: Same as section 3.35; this rounds out the three practical detection techniques (ping, DNS, ARP methods) that pair with the general detection approaches in section 3.34.

EXAM FOCUS:

- Keyword "only the promiscuous machine already has the ARP information cached", "responds without probing first" -> the ARP Method.
- Must-memorize set: the exam loves listing all three sniffer detection techniques together (Ping Method, DNS Method, ARP Method) and asking which one matches a given scenario description.

- Trap: the ARP Method and the Ping Method both use a ping message, but for different reasons: the Ping Method tests MAC address filtering behavior directly; the ARP Method tests whether the responding machine already secretly knew information it should not have had.

Memory trick: "P-D-A: Ping (wrong MAC), DNS (unwanted lookups), ARP (already knows, no need to ask)." Three techniques, three different telltale behaviors.

4. Tool Encyclopedia

4.1 Attack and flooding tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
macof	Floods a switch's CAM table with random MAC/IP pairs	Linux/Unix, dsniff suite	Sniffing	"floods the CAM table"
Yersinia	Multi-protocol Layer 2 attack tool (DHCP, STP, CDP, DTP, more)	Linux	Sniffing	"DHCP starvation tool"
dhcpStarvation.py	Scripted DHCP starvation attack	Python/github	Sniffing	"leases all DHCP addresses"
Hyenae	Flexible packet generator/flooder	Cross-platform	Sniffing	"DHCP starvation"
DHCPig	DHCP exhaustion attack tool	Python/github	Sniffing	"DHCP starvation"
arpspoof	Redirects packets by forging ARP replies	Linux, dsniff suite	Sniffing	"forges ARP replies"
Habu	Toolkit: ARP poisoning, sniffing, DHCP starvation	Python/github	Sniffing	"ARP poisoning toolkit"
bettercap	Full network attack and monitoring framework	Cross-platform	Sniffing	"MITM framework"
Ettercap	Comprehensive MITM attack suite	Cross-platform	Sniffing	"comprehensive MITM suite"

4.2 Detection, analysis, and defense tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
Wireshark	Packet capture and protocol analysis, display filters	Cross-platform	Sniffing / detection	"display filter"
Capsa Portable Network Analyzer	Portable capture, detects ARP poisoning/ flooding, finds source	Windows	Sniffing / detection	"locate the attack source"
OmniPeek	Packet sniffer with a Google Map view of public IP locations	Windows	Sniffing / detection	"Google Map in capture window"
OpUtils	Network and IP address management, ARP detection	ManageEngine	Detection	"ARP spoofing detection"
netspionage	ARP/network reconnaissance and detection helper	github	Detection	"ARP spoofing detection"
NetProbe	Network/ARP monitoring tool	github	Detection	"ARP spoofing detection"
ARP-GUARD	Dedicated ARP spoofing/flooding protection appliance	Commercial	Detection	"ARP spoofing detection"
RITA (Real Intelligence Threat Analytics)	Analyzes network traffic for beaconing/C2 patterns	github	Detection	"Real Intelligence Threat Analytics"
Observer Analyzer	Network performance and packet analysis	viavisolutions.com	Detection	"network analyzer"
PRTG Network Monitor	Network monitoring dashboard (bandwidth, uptime)	paessler.com	Monitoring	"network monitoring"
Network Performance Monitor (NPM)	Network monitoring dashboard	solarwinds.com	Monitoring	"network performance monitor"

4.3 Look-alike tools and how to pick

- **macof vs arpspoof:** macof attacks the switch's CAM table (Layer 2 switch memory) to force broadcast behavior. arpspoof attacks a host's or switch's ARP cache to misdirect specific traffic. If the question says "fills the switch's table", pick macof; if it says "forges ARP replies", pick arpspoof.
- **Ettercap/bettercap vs arpspoof:** arpspoof is narrow and does only ARP spoofing. Ettercap and bettercap are broad frameworks that can do ARP spoofing plus many other MITM techniques, sniffing, and plugins.
- **Capsa/OmniPeek vs Wireshark:** all three can capture and inspect packets. Capsa adds purpose-built ARP poisoning/flooding detection with source location. OmniPeek adds a geographic map overlay. Wireshark is the general-purpose, protocol-deep, filter-driven standard.
- **PRTG/NPM vs Capsa/OmniPeek/Wireshark:** PRTG and NPM are performance/uptime monitoring dashboards; they do not decode packet contents the way a sniffer does. If the question asks about reading actual packet data, do not pick PRTG or NPM.
- **Yersinia vs Hyenae/DHCPig/dhcpStarvation.py:** Yersinia is a multi-protocol Swiss-army-knife (DHCP, STP, CDP, DTP, HSRP, and more). The other three are single-purpose DHCP starvation tools.

5. Command Sheet

5.1 Cisco IOS: Port Security (MAC flooding and DHCP starvation defense)

Purpose: Limit how many MAC addresses a switch port will learn, and decide what happens when that limit is exceeded.

Full syntax (per interface, configuration mode):

```
interface <type> <slot/port>
 switchport port-security
 switchport port-security maximum <1-3072>
 switchport port-security violation {protect |
   restrict | shutdown}
 switchport port-security aging time <minutes>
 switchport port-security aging type
   {absolute | inactivity}
```

Flag table:

Flag/keyword	Meaning	When to use	Example
`maximum <n>`	Max MAC addresses allowed on the port (1 to 3072)	Set to 1 for a single known device	`switchport port-security maximum 1`
`violation restrict`	Drop violating traffic, log it, keep port up	Want visibility without an outage	`switchport port-security violation restrict`
`violation shutdown` (ADDED CONTEXT)	Err-disable the port on violation	Want the strictest response	`switchport port-security violation shutdown`
`aging time <n>`	Minutes before a learned MAC can age out	Allow devices to rotate over time	`switchport port-security aging time 2`
`aging type inactivity`	Age out only if the MAC goes quiet	Avoid aging out an active device	`switchport port-security aging type inactivity`

Demo commands (realistic scenario: lock a user access port to one device):

```
Switch(config)# interface gigabitethernet0/1
Switch(config-if)# switchport mode access
Switch(config-if)# switchport port-security
Switch(config-if)# switchport port-security maximum 1
Switch(config-if)# switchport port-security violation
  restrict
```

Sample output and how to read it:

```
Switch# show port-security interface gi0/1
Port Security              : Enabled
Port Status                : Secure-up
Violation Mode              : Restrict
Maximum MAC Addresses       : 1
Total MAC Addresses         : 1
Configured MAC Addresses    : 0
Security Violation Count    : 0
```

Read it top to bottom: "Enabled" confirms the feature is on; "Secure-up" means the port is active and compliant; "Violation Mode: Restrict" confirms it will drop extra MACs without shutting the port; "Total MAC Addresses: 1" of a "Maximum: 1" means the port is at its limit with no room for a second (rogue) device; "Security Violation Count: 0" means no attack has been attempted yet.

Common mistakes: forgetting `switchport mode access` first (port security requires a static access or trunk mode, not the default dynamic negotiation mode); setting `maximum` too high, which defeats the point of the control; choosing `violation shutdown` without a plan for who re-enables the port, which can cause unplanned outages.

What the exam asks about it: matching the command to "limit MAC addresses per port" or "defend against MAC flooding/DHCP starvation", and reading `show port-security` style output to say whether a port is currently compliant or already at its MAC limit.

5.2 Cisco IOS: DHCP Snooping

Purpose: Only trust DHCP server responses on specific, designated switch ports, and build the binding table that Dynamic ARP Inspection and IP Source Guard rely on.

Full syntax:

```
ip dhcp snooping
ip dhcp snooping vlan <vlan-id>
interface <type> <slot/port>
 ip dhcp snooping trust
```

Flag/keyword table:

Flag/keyword	Meaning	When to use	Example
`ip dhcp snooping`	Globally enables DHCP snooping	Always first step	`ip dhcp snooping`
`vlan <id>`	Chooses which VLAN(s) are snooped	Apply to every VLAN with end users	`ip dhcp snooping vlan 10`
`trust`	Marks an interface as trusted (DHCP server responses allowed)	Only on the uplink to the real DHCP server	`ip dhcp snooping trust`

Demo commands (realistic scenario: enable snooping on VLAN 10, trust only the uplink):

```
Switch(config)# ip dhcp snooping
Switch(config)# ip dhcp snooping vlan 10
Switch(config)# interface gigabitethernet0/24
Switch(config-if)# ip dhcp snooping trust
```

Sample output and how to read it:

```
Switch# show ip dhcp snooping
Switch DHCP snooping is enabled
DHCP snooping is configured on following VLANs:
10
DHCP snooping is configured on the following L3
Interfaces:

Interface      Trusted  Rate limit (pps)
-----------------------------------------
Gi0/24         yes      unlimited
```

Read it top to bottom: "enabled" confirms the global state; the VLAN list confirms which VLANs are protected; the Interface table shows exactly which port is trusted ("yes") and whether a rate limit is applied. Any port not listed as trusted here is untrusted by default and will have its DHCP server responses dropped.

Common mistakes: forgetting to mark the uplink as trusted (this breaks DHCP for the entire VLAN, since legitimate offers get dropped too); trusting an access port by mistake, which reopens the door to rogue DHCP servers.

What the exam asks about it: recognizing that all ports are untrusted by default, and reading `show ip dhcp snooping` / `show ip dhcp snooping binding` output to identify whether a specific port is trusted or to find a specific host's recorded binding.

5.3 Cisco IOS: Dynamic ARP Inspection (DAI)

Purpose: Validate ARP packets against the DHCP snooping binding table and discard ARP packets whose claimed MAC/IP pair does not match a known good binding.

Full syntax:

```
ip arp inspection vlan <vlan-id>
```

Demo command and sample output:

```
Switch(config)# ip arp inspection vlan 10
Switch# show ip arp inspection
```

```
Source Mac Validation      : Disabled
Destination Mac Validation : Disabled
IP Address Validation      : Disabled

Vlan  Configuration  Operation  ACL Match  Static ACL
10    Enabled        Active     -          No

Vlan  ACL Logging  DHCP Logging  Probe Logging
10    Deny         Deny          Off

Vlan  Forwarded  Dropped  DHCP Drops  ACL Drops
10    0          0        0           0

Vlan  DHCP Permits  ACL Permits  Probe Permits  Source MAC Failures
10    0             0            0              0

Vlan  Dest MAC Failures  IP Validation Failures  Invalid Protocol Data
10    0                  0                       0
```

Read it top to bottom: the top three lines show whether extra, optional MAC/IP validation checks are turned on (they are "Disabled" here, meaning DAI is only doing its core job). "Configuration: Enabled" and "Operation: Active" confirm DAI is actually

running on VLAN 10. The counter rows (Forwarded, Dropped, DHCP Drops, ACL Drops, and the failure counters) are all zero in this fresh example; after an attack attempt, "Dropped" and "Source MAC Failures" are the first counters that should increase.

Common mistakes: enabling DAI on a VLAN before DHCP snooping has had time to populate the binding table, which can cause legitimate hosts to be blocked; forgetting that DAI is per-VLAN, so a VLAN not listed is not protected.

What the exam asks about it: reading the `show ip arp inspection` output to say whether DAI is active, and recognizing the `%SW_DAI-4-DHCP_SNOOPING_DENY` syslog message as evidence of a blocked spoofed ARP packet.

5.4 Cisco IOS: BPDU Guard, Root Guard, Loop Guard, and UDLD (STP defense)

Purpose: Prevent a rogue switch from influencing or taking over the spanning tree, and detect broken unidirectional links.

Full syntax:

```
interface <type> <slot/port>
 spanning-tree portfast bpduguard
 spanning-tree guard root
 spanning-tree guard loop
 udld {enable | disable | aggressive}
```

Flag table:

Flag/keyword	Meaning	When to use	Example
`spanning-tree portfast bpduguard`	Disables the port if any BPDU arrives	Edge/access ports, where no BPDU should ever appear	`spanning-tree portfast bpduguard`
`spanning-tree guard root`	Blocks a port from ever being used to become the new root bridge	Ports facing untrusted or non-core switches	`spanning-tree guard root`
`spanning-tree guard loop`	Stops a port wrongly going to forwarding, causing a loop	Non-edge, redundant links	`spanning-tree guard loop`
`udld aggressive`	Probes a link, disables it if unidirectional	Fiber or long trunk links	`udld aggressive`

> **TRAP:** The screenshot on page 13 shows the same command text, `spanning-tree portfast bpduguard`, listed under both the BPDU Guard and Loop Guard headings. This looks like a slide duplication or transcription error. The command shown in this Command Sheet for Loop Guard (`spanning-tree guard loop`) is the standard Cisco IOS command for that feature and is marked "verify" because it was not clearly legible in the source screenshot.

Demo commands (realistic scenario: harden an access port and a trunk link):

```
Switch(config)# interface fastethernet0/5
Switch(config-if)# spanning-tree portfast
Switch(config-if)# spanning-tree portfast bpduguard
Switch(config)# interface gigabitethernet0/1
Switch(config-if)# spanning-tree guard root
```

Sample output and how to read it:

```
Switch# show spanning-tree interface fa0/5 detail
Port 5 (FastEthernet0/5) is err-disabled
 Port Guard is bpdu guard
```

Read it as: the port has been shut down ("err-disabled") because it received a BPDU it should never have seen on an edge port, and the log line "Port Guard is bpdu guard" names exactly which protection triggered the shutdown.

Common mistakes: applying BPDU Guard to a port that legitimately connects to another switch (this will disable a valid trunk); confusing Root Guard (blocks becoming root via this port) with BPDU Guard (blocks BPDUs entirely on this port).

What the exam asks about it: matching each STP defense command to the specific failure it prevents, and reading an err-disabled port's reason code to identify which guard feature triggered it.

5.5 Wireshark Display Filters

Purpose: Narrow the view of an already-captured file down to the packets relevant to an investigation.

Full syntax (typed into the filter bar, then press Enter):

```
<protocol>
<field> == <value>
<field> == <value> && <field> == <value>
<field> == <value> or <field> == <value>
```

Flag/operator table:

Operator	Meaning	When to use	Example
(protocol name)	Show only that protocol's packets	Quick protocol-only view	`arp`
`==`	Equals	Match an exact field value	`tcp.port==23`
`&&`	Logical AND	Combine two required conditions	`ip.addr==192.168.1.100 && tcp.port==23`

Operator	Meaning	When to use	Example
`or`	Logical OR	Match either of two conditions	`ip.addr==10.0.0.4 or ip.addr==10.0.0.5`
`>` / `<`	Greater than / less than	Numeric range filtering	`frame.number > 15 && frame.number < 30`

Demo commands (realistic scenario: hunt for Telnet and ICMP activity involving one host):

```
tcp.port==23
ip.addr==10.0.1.12 && icmp && frame.number > 15
  && frame.number < 30
ip.src==205.153.63.30 or ip.dst==205.153.63.30
```

Sample output and how to read it: after typing `tcp.port==23` and pressing Enter, Wireshark's packet list instantly shrinks to show only frames where either the source or destination TCP port is 23 (Telnet); the bar turns green when the filter syntax is valid, and the status bar at the bottom reports how many packets out of the total now match, confirming the filter is actually narrowing the view rather than showing everything.

Common mistakes: using `=` instead of `==` (single equals is invalid in display filter syntax and the filter bar turns red/pink); forgetting that a display filter changes what you see, not what was captured, so it cannot reveal packets that were never captured in the first place.

What the exam asks about it: reading a given filter string and saying what traffic it will show, or being given a plain-English requirement and picking the matching filter syntax.

6. Ports, Protocols, Standards and Acronyms

6.1 Ports and protocols seen in the screenshots

Protocol/Port	Simple purpose	Default port or layer	Known weakness	Related attack or tool
Telnet	Remote terminal login	TCP 23	Sends credentials in cleartext	Sniffing, Wireshark filter `tcp.port==23`
FTP	File transfer	TCP 20/21	Sends credentials and data in cleartext	Sniffing
POP3	Email retrieval	TCP 110	Cleartext login by default	Sniffing (Data Link layer slide)

Protocol/Port	Simple purpose	Default port or layer	Known weakness	Related attack or tool
IMAP	Email retrieval and sync	TCP 143	Cleartext login by default	Sniffing (Data Link layer slide)
SSL/TLS	Encrypts traffic between client and server	TCP varies (for example 443 with HTTPS)	Still visible at the Data Link layer, but payload is encrypted	Named on the Data Link layer slide
SSH	Encrypted remote login and file transfer	TCP 22	Strong when configured correctly	Replacement for Telnet
DNS	Resolves domain names to IP addresses	UDP/TCP 53	No built-in response authentication (without DNSSEC)	DNS poisoning (section 3.25-3.29)
DHCP	Assigns IP configuration automatically	UDP 67 (server), UDP 68 (client)	No built-in server authentication	Rogue DHCP, DHCP starvation
ARP	Resolves IP addresses to MAC addresses	Data Link layer (no port, no IP header)	Stateless, no authentication	ARP spoofing/ poisoning
ICMP	Network diagnostics (ping, router discovery)	Network layer (no port)	Can be spoofed; IRDP rides on ICMP	IRDP spoofing, Ping Method detection
STP	Prevents Layer 2 loops by electing a root bridge	Data Link layer (no port)	Lowest priority wins root election, even a rogue switch	STP attack
802.1Q (VLAN tagging)	Tags frames with a VLAN ID across trunks	Data Link layer	Double tagging abuses the native VLAN's untagged handling	VLAN hopping (double tagging)

6.2 Standards and acronyms

Acronym	Full name	Why it matters here
CAM	Content Addressable Memory	The switch's MAC-to-port lookup table, the target of MAC flooding
OUI	Organizationally Unique Identifier	The first 3 bytes of a MAC address, identifying the manufacturer
NIC	Network Interface Card	The hardware/driver that can be set to promiscuous mode

Acronym	Full name	Why it matters here
SPAN	Switched Port Analyzer	A switch port configured to mirror all traffic
DAI	Dynamic ARP Inspection	Validates ARP packets against the DHCP snooping binding table
BPDU	Bridge Protocol Data Unit	The STP control message that BPDU Guard watches for
UDLD	Unidirectional Link Detection	Detects links that only work in one direction
VLAN	Virtual Local Area Network	A logical segmentation boundary that VLAN hopping bypasses
IRDP	ICMP Router Discovery Protocol	Lets hosts learn routers via ICMP advertisements; can be spoofed
DNSSEC	Domain Name System Security Extension	Lets a resolver cryptographically verify a DNS response's source
NXDOMAIN	Non-Existent Domain	"No such domain" reply; rate limit it to block poisoning
ACL	Access Control List	Restricts access to a defined, trusted range of IP addresses
MITM	Man-in-the-Middle	Attacker secretly relays or alters traffic between two parties
C2	Command and Control	Remote server malware calls back to (ADDED, see RITA)

ADDED CONTEXT: The module does not name a specific port for ARP or STP because both operate at the Data Link layer, below the Network layer where port numbers exist. Questions that ask for "the port used by ARP" are testing whether you know ARP has no port at all.

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attack and its countermeasure

Attack	Countermeasure
MAC flooding (macof, CAM table overflow)	Port security with a MAC address maximum per port

Attack	Countermeasure
DHCP starvation	Port security (MAC limit on edge ports)
Rogue DHCP server	DHCP snooping (trust only the real server's port)
ARP spoofing/poisoning	Dynamic ARP Inspection using the DHCP snooping binding table
MAC spoofing/duplicating	DHCP snooping binding table, DAI, and IP Source Guard together
IRDP spoofing	Disable IRDP where not needed; use static/ authenticated routing
VLAN hopping: switch spoofing	`switchport mode access` and `switchport nonegotiate` on trunks
VLAN hopping: double tagging	Move the native VLAN off the default and tag it explicitly
STP attack	BPDU Guard, Root Guard, Loop Guard, UDLD
Intranet DNS spoofing	Defend the underlying ARP poisoning (DAI); local DNS resolution
Internet DNS spoofing	Anti-malware to stop the Trojan; secure DNS config
Proxy server DNS poisoning	Anti-malware; lock down browser proxy settings
DNS cache poisoning	DNSSEC, random source ports, NXDOMAIN rate limiting
General sniffing	Encryption end to end, switches, static ARP/IP, promiscuous checks

7.2 Look-alike pairs

- **Passive vs Active sniffing.** Difference: passive needs no injected packets (hub); active injects packets to force a switch to leak traffic. Tell them apart: does the scenario mention a hub (passive) or a switch-breaking technique (active)?
- **ARP spoofing vs MAC spoofing.** Difference: ARP spoofing lies about an IP-to-MAC mapping; MAC spoofing claims to literally be a specific other real device's MAC. Tell them apart: does the attack forge an ARP reply (ARP spoofing), or change the attacker's own NIC address to match a victim's (MAC spoofing)?
- **MAC flooding vs ARP spoofing.** Difference: MAC flooding overflows the switch's CAM table to force broadcast; ARP spoofing poisons specific ARP caches with false mappings. Tell them apart: is the target the switch's table (flooding) or a host's/switch's ARP cache entries (spoofing)?
- **DHCP starvation vs Rogue DHCP server.** Difference: starvation is a DoS that exhausts the address pool; a rogue server hands out malicious configuration. Tell them apart: does the victim get no address at all (starvation), or a bad gateway/ DNS/IP (rogue server)?

- **Switch spoofing vs Double tagging (VLAN hopping).** Difference: switch spoofing tricks a switch into forming a trunk; double tagging smuggles a frame across VLANs using two stacked tags. Tell them apart: is a rogue switch negotiating a trunk (switch spoofing), or is one frame carrying two VLAN tags (double tagging)?
- **Intranet vs Internet DNS spoofing.** Difference: intranet requires the attacker on the LAN and uses ARP poisoning; internet uses a Trojan to change the victim's DNS server remotely. Tell them apart: is the attacker physically local (intranet), or using malware with no local access needed (internet)?
- **Internet DNS spoofing vs Proxy server DNS poisoning.** Difference: internet spoofing changes the DNS server setting; proxy poisoning changes the browser's proxy setting. Tell them apart: which setting did the Trojan change, the DNS server address or the proxy address?
- **DNS cache poisoning vs the other three DNS attacks.** Difference: cache poisoning corrupts a shared resolver, affecting every later user of it; the other three target one victim at a time. Tell them apart: does the attack affect only one machine, or everyone who later queries the same resolver?
- **BPDU Guard vs Root Guard.** Difference: BPDU Guard disables a port the instant any BPDU appears at all; Root Guard only blocks that port from becoming the path to a new root. Tell them apart: should this port never see a BPDU at all (BPDU Guard, edge ports), or might it legitimately see BPDUs but never a new root claim (Root Guard, non-edge ports)?
- **Capsa/OmniPeek vs PRTG/NPM.** Difference: Capsa and OmniPeek decode packet contents (deep packet sniffers); PRTG and NPM track uptime and bandwidth (monitoring dashboards). Tell them apart: does the tool need to read inside packets (sniffer), or just track metrics over time (monitor)?
- **Ping Method vs ARP Method (sniffer detection).** Difference: the Ping Method tests whether a NIC rejects frames with a mismatched MAC; the ARP Method tests whether a machine already secretly knew information it should not have had. Tell them apart: is the test about MAC filtering behavior (Ping Method), or prior knowledge the machine should not have (ARP Method)?

8. Exam Cheat Sheet

8.1 Keyword to answer

Keyword in the question	Likely answer
"stateless protocol", "IP to MAC"	ARP
"fixed-size dynamic table", "MAC to port"	CAM table
"fills the CAM table", "dsniff"	MAC flooding / macof
"exhausts the DHCP scope"	DHCP starvation
"attacker becomes the gateway/DNS server"	Rogue DHCP server attack

Keyword in the question	Likely answer
"forges ARP replies"	ARP spoofing/poisoning, arpspoof
"re-uses a legitimate client's MAC"	MAC spoofing/duplicating
"bypasses MAC filtering on a Wi-Fi AP"	MAC spoofing/duplicating
"ICMP Router Discovery Protocol"	IRDP spoofing
"trunk link formed by trick", "rogue switch negotiates a trunk"	VLAN hopping: switch spoofing
"two stacked 802.1Q tags"	VLAN hopping: double tagging
"priority 0 becomes root bridge"	STP attack
"BPDU on an edge port"	BPDU Guard
"blocks a port from becoming the path to the root"	Root Guard
"unidirectional link"	UDLD
"attacker on the LAN, uses ARP poisoning for DNS"	Intranet DNS spoofing
"Trojan changes the DNS server setting"	Internet DNS spoofing
"Trojan changes the browser proxy"	Proxy server DNS poisoning
"forged records in a resolver's cache, affects many users"	DNS cache poisoning
"cryptographically verify DNS responses"	DNSSEC
"Google Map of captured public IPs"	OmniPeek
"detects ARP poisoning/flooding and locates the source"	Capsa Portable Network Analyzer
"wrong MAC, right IP, only a sniffer answers"	Ping Method (sniffer detection)
"unexpected reverse DNS lookup traffic"	DNS Method (sniffer detection)
"already knows, answers without ARP-probing first"	ARP Method (sniffer detection)

8.2 Must-memorize lists

- Six active sniffing techniques: MAC Flooding, DHCP Attacks, DNS Poisoning, Switch Port Stealing, ARP Poisoning, Spoofing Attack.
- Four DNS poisoning types, in module order: Intranet, Internet, Proxy Server, Cache Poisoning.
- Four STP defenses: BPDU Guard, Root Guard, Loop Guard, UDLD.
- Three sniffer detection techniques: Ping Method, DNS Method, ARP Method.
- Three detection approaches (broader category): check promiscuous mode, run IDS, run network tools.
- Fifteen general anti-sniffing countermeasures, grouped as: reduce physical/broadcast exposure, encrypt everything, lock down addressing, actively look for sniffers.

- Fourteen DNS spoofing defenses, with DNSSEC as the single most important one.

8.3 Numbers and defaults

Item	Value
MAC address length	48 bits (6 bytes): 3-byte OUI + 3-byte NIC-specific
Port security maximum MAC range	1 to 3072
STP default bridge priority (example in slide)	32768 or 32769, beaten by an attacker's priority 0
Telnet port	TCP 23
DHCP server/client ports	UDP 67 (server) / UDP 68 (client)
DNS port	UDP/TCP 53

8.4 Rules for picking the BEST option

- If two attacks both "redirect traffic to the attacker", check whether the mechanism is Layer 2 (ARP/MAC spoofing, STP, VLAN hopping) or DNS- level (the four DNS poisoning types); pick based on exactly what was manipulated, not just the end result.
- If a defense question lists both a narrow fix (for example port security) and a broader fix (for example DHCP snooping plus DAI), pick the option that matches the *specific* attack named in the question, not just any plausible-sounding security control.
- If a question asks "best next step to confirm a suspected sniffer", prefer an active detection technique (Ping, DNS, or ARP Method) over a passive prevention measure (encryption), since the question is about detection, not prevention.
- When DNSSEC and SSL/HTTPS both appear as options for a DNS poisoning question, remember DNSSEC fixes the DNS resolution integrity problem itself, while SSL/HTTPS only protect the data in transit after resolution; pick DNSSEC when the question asks how to stop the poisoning itself.

9. Practice Questions

These 26 questions are original. Each answer and explanation sits directly below its question.

Question 1. A penetration tester plugs a laptop into an open network port on an old factory floor where devices are still connected via a hub. The tester captures full network traffic without running any extra tools or sending any additional packets. What type of sniffing is this?

- A. Active sniffing

- B. Passive sniffing
- C. ARP poisoning
- D. MAC flooding

Answer: B. Passive sniffing works on a hub because every packet is already sent to every port, with no extra packets needed.

- A is wrong: active sniffing requires injecting packets, which did not happen here.
- C is wrong: ARP poisoning is a specific active technique, not what happened on a hub.
- D is wrong: MAC flooding targets a switch's CAM table; there is no switch described here.

Question 2. Which OSI layer does a sniffer operate at, making the upper layers unaware that sniffing has occurred?

- A. Application layer
- B. Network layer
- C. Data Link layer
- D. Transport layer

Answer: C. The module states sniffers operate at the Data Link layer, below all the layers that carry application data.

- A is wrong: the Application layer is one of the layers that is unaware, not the layer doing the sniffing.
- B is wrong: Network layer protocols (like IP) sit above Data Link and are also unaware.
- D is wrong: Transport layer (like TCP/UDP) also sits above Data Link.

Question 3. An attacker runs a tool from the dsniff suite that sends a huge number of frames with random source MAC addresses, overflowing a switch's table so it starts broadcasting to every port. Which tool and which switch structure are being described?

- A. arpspoof; ARP cache
- B. macof; CAM table
- C. Yersinia; STP table
- D. Ettercap; routing table

Answer: B. macof is the dsniff tool for this, and it targets the switch's CAM (Content Addressable Memory) table.

- A is wrong: arpspoof forges ARP replies; it does not flood the CAM table with random MACs.
- C is wrong: Yersinia is used for DHCP/STP-style attacks, not this CAM flooding technique.

- D is wrong: Ettercap is a broader MITM suite, not the dsniff CAM- flooding tool, and switches do not have a "routing table".

Question 4. Which Cisco IOS command sets the maximum number of MAC addresses a port will learn, as a defense against MAC flooding?

- A. `ip dhcp snooping vlan 10`
- B. `switchport port-security maximum 1`
- C. `spanning-tree guard root`
- D. `ip arp inspection vlan 10`

Answer: B. This is the exact port security command that caps the number of MAC addresses, valid in the range 1 to 3072.

- A is wrong: this command enables DHCP snooping on a VLAN, unrelated to MAC address limits.
- C is wrong: this is an STP defense against root bridge takeover, not a MAC address limit.
- D is wrong: this enables Dynamic ARP Inspection, not a MAC address limit.

Question 5. A rogue DHCP server is handing out a bogus default gateway address to new clients. Which Cisco IOS feature specifically blocks this by only trusting DHCP server responses on designated ports?

- A. Port security
- B. BPDU Guard
- C. DHCP snooping
- D. IP Source Guard

Answer: C. DHCP snooping only accepts DHCP server messages (offers, acks) from ports explicitly marked trusted, directly blocking a rogue server on any other port.

- A is wrong: port security limits MAC address count per port; it does not inspect DHCP server message content.
- B is wrong: BPDU Guard defends against STP attacks, unrelated to DHCP.
- D is wrong: IP Source Guard checks ordinary traffic against the binding table; it is a complement to DHCP snooping, not the DHCP- specific trust mechanism itself.

Question 6. During an internal test, an attacker broadcasts forged DHCP requests fast enough that the entire DHCP scope is leased out before any real employee laptop can get an address. What is this attack called?

- A. Rogue DHCP server attack
- B. DHCP starvation attack
- C. ARP poisoning
- D. VLAN hopping

Answer: B. Exhausting the DHCP address pool with forged requests is the definition of DHCP starvation, a denial-of-service attack.

- A is wrong: a rogue server hands out bad configuration; this question describes exhausting the pool, not handing out bad settings.
- C is wrong: ARP poisoning targets ARP caches, not DHCP leases.
- D is wrong: VLAN hopping is about bypassing VLAN segmentation, unrelated to DHCP leasing.

Question 7. Which tool is specifically named in the module as a DHCP starvation attack tool that also supports attacks against STP, CDP, and other Layer 2 protocols?

- A. macof
- B. Yersinia
- C. arpspoof
- D. OmniPeek

Answer: B. Yersinia is the multi-protocol Layer 2 attack framework named for DHCP starvation in the module, and it also supports STP, CDP, and other protocol attacks.

- A is wrong: macof is for CAM table/MAC flooding, not DHCP or STP.
- C is wrong: arpspoof is single-purpose, for ARP spoofing only.
- D is wrong: OmniPeek is a sniffer/analysis tool, not an attack tool.

Question 8. An ARP reply arrives claiming a certain IP address belongs to a MAC address that does not match the switch's DHCP snooping binding table entry for that interface. Which feature discards this packet?

- A. Port security
- B. BPDU Guard
- C. Dynamic ARP Inspection (DAI)
- D. UDLD

Answer: C. DAI checks ARP packets against the DHCP snooping binding table and discards any that do not match.

- A is wrong: port security counts MAC addresses per port; it does not validate ARP packet content against IP bindings.
- B is wrong: BPDU Guard reacts to STP control messages, not ARP.
- D is wrong: UDLD detects unidirectional links, unrelated to ARP validation.

Question 9. An attacker sniffs the network, finds the MAC address of a currently connected, authorized laptop, and then configures their own NIC to use that exact same MAC address to gain network access. What is this attack called?

- A. ARP spoofing
- B. MAC spoofing/duplicating
- C. IRDP spoofing

- D. STP attack

Answer: B. Reusing another client's real MAC address to take over their identity on the network is MAC spoofing/duplicating.

- A is wrong: ARP spoofing forges IP-to-MAC mapping claims; it does not require literally copying another device's real MAC address.
- C is wrong: IRDP spoofing is about forging router advertisements, not copying a client's MAC.
- D is wrong: STP attacks are about becoming the root bridge, unrelated to copying a MAC address.

Question 10. Which of the following attacks can also be used to bypass a wireless access point's MAC address filtering, according to the module?

- A. STP attack
- B. VLAN hopping
- C. MAC spoofing/duplicating
- D. DNS cache poisoning

Answer: C. The module specifically notes that MAC spoofing/ duplicating can be used to bypass MAC filtering on wireless access points.

- A is wrong: STP attacks target the spanning tree election, not Wi-Fi MAC filters.
- B is wrong: VLAN hopping bypasses VLAN segmentation, not Wi-Fi MAC filtering specifically.
- D is wrong: DNS cache poisoning affects DNS resolution, not MAC-based access control.

Question 11. A host's routing table suddenly changes its default router after receiving an unsolicited advertisement message over ICMP. Which attack best matches this description?

- A. ARP spoofing
- B. IRDP spoofing
- C. MAC flooding
- D. Rogue DHCP server attack

Answer: B. IRDP spoofing sends a forged ICMP Router Discovery Protocol advertisement to change a host's default router.

- A is wrong: ARP spoofing changes IP-to-MAC mappings, not the default router setting directly via ICMP router discovery.
- C is wrong: MAC flooding targets a switch's CAM table, not a host's routing table.
- D is wrong: a rogue DHCP server changes the gateway via the DHCP protocol, not via ICMP router discovery advertisements.

Question 12. An attacker connects a rogue switch to a network port and successfully tricks the legitimate switch into forming an unauthorized trunk link with it. Which VLAN hopping method is this?

- A. Double tagging
- B. Switch spoofing
- C. STP attack
- D. DAI bypass

Answer: B. Tricking a legitimate switch into forming a trunk link is switch spoofing, one of the two VLAN hopping methods.

- A is wrong: double tagging uses two stacked 802.1Q tags on a frame, not a tricked trunk negotiation.
- C is wrong: an STP attack is about becoming the root bridge, a separate attack from VLAN hopping.
- D is wrong: "DAI bypass" is not a named VLAN hopping method in this module.

Question 13. Which single Cisco IOS command setting helps defend specifically against the double tagging form of VLAN hopping by moving the native VLAN away from its default?

- A. `switchport mode access`
- B. `switchport nonegotiate`
- C. `switchport trunk native vlan 999`
- D. `switchport port-security maximum 1`

Answer: C. Changing the native VLAN on trunk ports to an unused ID directly defends against double tagging, which relies on the default native VLAN being predictable and untagged.

- A is wrong: this defends against switch spoofing (locks a port as access-only), not double tagging specifically.
- B is wrong: this also defends against switch spoofing, by stopping trunk negotiation.
- D is wrong: this is a MAC flooding/port security command, unrelated to VLAN hopping.

Question 14. A rogue switch on the network advertises a bridge priority of 0, causing it to become the new root bridge and pull all traffic through itself. Which combination of features would best have prevented this?

- A. DHCP snooping and IP Source Guard
- B. Root Guard and BPDU Guard
- C. DNSSEC and SSL
- D. Port security and ACLs

Answer: B. Root Guard stops a port from becoming the path to a new root bridge, and BPDU Guard disables ports that should never see a BPDU at all; together they are the module's named STP attack defenses.

- A is wrong: DHCP snooping and IP Source Guard defend against DHCP and IP/MAC spoofing attacks, not STP root bridge takeover.
- C is wrong: DNSSEC and SSL defend DNS integrity and data-in-transit confidentiality, unrelated to STP.
- D is wrong: port security and ACLs limit MAC addresses and IP access, not STP election behavior.

Question 15. An attacker on the same LAN as the victim uses ARP poison routing to redirect the victim's DNS requests to the attacker's own machine, where a fake website is already set up. Which DNS poisoning type is this?

- A. Internet DNS spoofing
- B. Proxy server DNS poisoning
- C. Intranet DNS spoofing
- D. DNS cache poisoning

Answer: C. Intranet DNS spoofing specifically requires the attacker to be on the LAN and works well when paired with ARP poison routing.

- A is wrong: Internet DNS spoofing uses a Trojan and does not require the attacker to be physically on the LAN.
- B is wrong: Proxy server DNS poisoning changes the victim's browser proxy setting, not the DNS resolution path via ARP poisoning.
- D is wrong: DNS cache poisoning targets a shared resolver's cache, affecting many users, not just one LAN-local victim.

Question 16. A Trojan silently changes a victim's configured DNS server address to an attacker-controlled IP, with no need for the attacker to be on the same local network. Which attack is this?

- A. Intranet DNS spoofing
- B. Internet DNS spoofing
- C. Proxy server DNS poisoning
- D. ARP spoofing

Answer: B. Internet DNS spoofing uses a Trojan to change the DNS server setting, and it does not require local network access.

- A is wrong: Intranet DNS spoofing requires the attacker to be on the LAN, unlike this scenario.
- C is wrong: Proxy server DNS poisoning changes the browser's proxy setting, not the DNS server setting.

- D is wrong: ARP spoofing forges ARP replies on the local network, not a remote DNS server change via Trojan.

Question 17. A Trojan changes a victim's Internet Explorer proxy setting so that all of the victim's web traffic is funneled through the attacker's machine before reaching any website. Which attack is this?

- A. Internet DNS spoofing
- B. Intranet DNS spoofing
- C. Proxy server DNS poisoning
- D. DNS cache poisoning

Answer: C. This is Proxy Server DNS Poisoning, which despite its name works entirely by changing the browser's proxy setting, not by poisoning any DNS record.

- A is wrong: Internet DNS spoofing changes the DNS server setting, not the proxy setting.
- B is wrong: Intranet DNS spoofing requires LAN access and ARP poisoning, not a Trojan-changed proxy setting.
- D is wrong: DNS cache poisoning corrupts a shared resolver's cache, a different mechanism entirely from changing a proxy setting.

Question 18. An internal DNS resolver accepts a forged response claiming to be from the authoritative server for a domain, and caches it. Every other user who later queries that same resolver for the same domain is redirected to a fake website. Which attack is this?

- A. Proxy server DNS poisoning
- B. DNS cache poisoning
- C. Intranet DNS spoofing
- D. IRDP spoofing

Answer: B. This is DNS cache poisoning: a forged record enters a shared resolver's cache and is then served to every later requester.

- A is wrong: proxy server poisoning affects one victim's browser setting, not a shared resolver used by many users.
- C is wrong: intranet DNS spoofing requires ARP poisoning and LAN access, and typically targets one victim, not a shared cache serving many users.
- D is wrong: IRDP spoofing changes a host's default router via ICMP, an unrelated mechanism to DNS resolver caching.

Question 19. Which single countermeasure from the module's 14-point DNS defense list most directly stops DNS cache poisoning by letting a resolver cryptographically verify a response's source?

- A. Use SSL for securing the traffic
- B. Implement DNSSEC

- C. Use SSH encryption
- D. Secure internal machines

Answer: B. DNSSEC is specifically designed to let a resolver verify that a DNS response truly came from the claimed authoritative source, directly addressing cache poisoning.

- A is wrong: SSL protects data in transit after resolution; it does not verify the DNS response itself.
- C is wrong: SSH protects a different kind of session traffic; it does not validate DNS responses.
- D is wrong: securing internal machines reduces malware risk generally, but is not the specific cryptographic DNS validation mechanism.

Question 20. Which Wireshark display filter correctly shows only the traffic between two specific IP addresses, 10.0.0.4 and 10.0.0.5?

- A. `ip.addr == 10.0.0.4 && ip.addr == 10.0.0.5`
- B. `ip.addr == 10.0.0.4 or ip.addr == 10.0.0.5`
- C. `tcp.port == 10.0.0.4`
- D. `ip.addr = 10.0.0.4`

Answer: B. Using `or` between the two IP address conditions correctly shows packets involving either address.

- A is wrong: using `&&` (AND) between two different exact IP address matches on the same field would match almost nothing, since a single packet usually cannot equal two different address values at once in the way this filter checks it.
- C is wrong: `tcp.port` expects a port number, not an IP address; this filter is invalid for this purpose.
- D is wrong: a single `=` is not valid Wireshark display filter syntax; it must be `==`.

Question 21. During an investigation, a security analyst pings a suspect host using its correct IP address but an intentionally incorrect destination MAC address. The host responds anyway. What does this most likely indicate?

- A. The host is behind a firewall
- B. The host's NIC is in promiscuous mode and likely running a sniffer
- C. The host has DHCP snooping enabled
- D. The host is running DNSSEC

Answer: B. A NIC that is not in promiscuous mode should reject frames with a mismatched MAC address; a response suggests the NIC accepted the frame anyway, consistent with promiscuous mode (the Ping Method of sniffer detection).

- A is wrong: a firewall rule is unrelated to whether a NIC accepts frames with the wrong destination MAC at the Ethernet layer.

- C is wrong: DHCP snooping is a switch feature about DHCP messages, not related to this MAC-mismatch ping test.
- D is wrong: DNSSEC is a DNS integrity feature, unrelated to Ethernet frame acceptance behavior.

Question 22. An analyst notices that one particular desktop performs reverse DNS lookups on IP addresses it was never directly communicating with. Which sniffer detection technique does this match?

- A. ARP Method
- B. Ping Method
- C. DNS Method
- D. Port security violation

Answer: C. Sniffers commonly perform reverse DNS lookups to identify hosts from captured IP addresses; unexpected reverse DNS traffic for unrelated addresses is the DNS Method of sniffer detection.

- A is wrong: the ARP Method is about a host already having cached ARP information it should not have, not about DNS lookup behavior.
- B is wrong: the Ping Method relies on sending a ping with a mismatched MAC address, not watching DNS lookup traffic.
- D is wrong: a port security violation is a switch-level MAC address limit event, unrelated to DNS lookup behavior.

Question 23. On a network segment, one machine responds immediately to a non-broadcast ping about a host it was never directly communicating with, without first sending its own ARP probe to find that host's MAC address. What does this suggest, and which technique identified it?

- A. The machine has a static ARP entry; Ping Method
- B. The machine is in promiscuous mode and already cached the needed ARP information; ARP Method
- C. The machine has DNSSEC enabled; DNS Method
- D. The machine has port security enabled; this is unrelated to sniffer detection

Answer: B. Only a machine in promiscuous mode would have already seen and cached ARP information for traffic not addressed to it, letting it skip the normal ARP probe step; this is the ARP Method of sniffer detection.

- A is wrong: a static ARP entry would not explain seeing information for a host that never directly communicated with this machine.
- C is wrong: DNSSEC is unrelated to ARP caching behavior.
- D is wrong: port security is a MAC-address-count control on switch ports, unrelated to this ARP caching behavior.

Question 24. Which tool, as named in the module, displays a Google Map in its capture window showing the physical locations of public IP addresses of captured packets?

- A. Capsa Portable Network Analyzer
- B. OmniPeek
- C. Wireshark
- D. PRTG Network Monitor

Answer: B. The module specifically describes OmniPeek's capture window as displaying a Google Map with the locations of captured public IP addresses.

- A is wrong: Capsa is described as a portable analyzer that detects ARP poisoning and flooding, not a map-based tool in this module.
- C is wrong: Wireshark is the general-purpose protocol analyzer named for its display filters, not a map feature in this module.
- D is wrong: PRTG is a network performance monitoring dashboard, not a packet sniffer with a map view.

Question 25. An attacker wants to use an AI assistant responsibly in a lab to speed up writing a report that explains why ARP is vulnerable to spoofing. Which prompt best fits a defensive, educational use consistent with this module's AI angle guidance?

- A. "Write me a fully working exploit to spoof ARP on a production network without permission."
- B. "Explain in two sentences why ARP has no built-in authentication, and give one Cisco IOS defense that fixes it."
- C. "Generate a phishing email pretending to be IT support."
- D. "Write malware that disables antivirus silently."

Answer: B. This prompt asks for an educational explanation and a defensive control, matching the module's guidance on using AI assistance for understanding concepts and writing reports, not for causing harm.

- A is wrong: this asks for an unauthorized exploit against a production network, which is out of scope for lawful, educational use.
- C is wrong: this requests a phishing lure, unrelated to understanding or defending against ARP spoofing, and is itself a harmful request.
- D is wrong: this requests malware creation, entirely unrelated to learning about ARP and clearly harmful.

Question 26. A defender wants an AI assistant to help triage a large Wireshark capture file after a suspected sniffing incident. Which prompt best matches the module's AI angle for defenders?

- A. "Decrypt this victim's banking password from the capture file."

- B. "Here is a list of HTTP Basic Auth headers I captured during an authorized test. Decode each one and list the usernames found so I can confirm which accounts need a forced password reset."
- C. "Write a tool that automatically exfiltrates captured credentials to my server."
- D. "Hide this capture file from the incident response team."

Answer: B. This matches the module's AI angle example of using an assistant to help interpret captured traffic from an authorized test, in support of a legitimate defensive action (forcing password resets).

- A is wrong: "decrypt a victim's banking password" implies unauthorized, harmful use against a real victim, not an authorized test or defensive triage.
- C is wrong: building an exfiltration tool is an offensive, harmful action, not a defensive triage task.
- D is wrong: hiding evidence from incident responders is obstruction, the opposite of the defensive, educational use the module's AI angle describes.

10. Flashcards

Q: What is packet sniffing? | A: Monitoring and capturing all data packets passing through a network using a software or hardware device. Q: What is promiscuous mode? | A: A NIC mode that accepts and reads every frame on the segment, not only frames addressed to it. Q: What is passive sniffing? | A: Sniffing on a hub, where traffic is already sent to every port with no extra packets needed. Q: What is active sniffing? | A: Sniffing on a switch by injecting packets (ARP, DHCP, STP, etc.) to force the switch to leak traffic. Q: Name the six active sniffing techniques. | A: MAC Flooding, DHCP Attacks, DNS Poisoning, Switch Port Stealing, ARP Poisoning, Spoofing Attack. Q: At which OSI layer does a sniffer operate? | A: The Data Link layer. Q: What is a SPAN port? | A: A switch port configured to receive a copy of every packet passing through the switch. Q: What is a CAM table? | A: A switch's fixed-size dynamic table that stores MAC addresses and their associated ports and VLANs. Q: How long is a MAC address, and how is it split? | A: 48 bits (6 bytes): a 3-byte OUI plus a 3-byte NIC-specific part. Q: What happens when a CAM table fills up? | A: The switch resets to a learning/flooding mode and broadcasts to every port like a hub. Q: What tool is named for MAC flooding? | A: macof, a Unix/Linux tool from the dsniff suite. Q: What Cisco command sets the max MAC addresses per port? | A: switchport port-security maximum <1-3072>. Q: What is a rogue DHCP server attack? | A: An attacker's DHCP server answers requests with bogus settings (wrong gateway, DNS, or IP). Q: What is DHCP starvation? | A: A DoS attack that leases out every address in the DHCP scope so real users cannot get one. Q: What tool is named for DHCP starvation? | A: Yersinia (also dhcpStarvation.py, Hyenae, DHCPig). Q: What Cisco feature stops rogue DHCP servers? | A: DHCP snooping, which only trusts DHCP server messages on trusted ports. Q: What is ARP spoofing/poisoning? | A: Forging ARP request/reply packets so a victim's ARP cache maps an IP to the attacker's MAC. Q: Name three ARP spoofing

tools. | A: arpspoof, Ettercap, bettercap (also Habu, RITM, ARP Spoofer). Q: What defends against ARP poisoning? | A: Dynamic ARP Inspection (DAI) using the DHCP snooping binding table. Q: What command enables DAI on a VLAN? | A: `ip arp inspection vlan <id>` Q: What is MAC spoofing/duplicating? | A: Reusing a legitimate, currently connected client's real MAC address to steal their network identity. Q: What extra use does MAC spoofing have against Wi-Fi? | A: It can bypass a wireless access point's MAC address filtering. Q: What defends against MAC spoofing? | A: DHCP snooping binding table, Dynamic ARP Inspection, and IP Source Guard together. Q: What is IRDP? | A: ICMP Router Discovery Protocol, which lets a host learn routers via advertisement/solicitation messages. Q: What is IRDP spoofing used for? | A: Man-in-the-middle, denial-of-service, and passive sniffing attacks, by changing a victim's default router. Q: What is VLAN hopping? | A: A method to bypass VLAN segmentation and gain unauthorized access to another VLAN's resources. Q: Name the two VLAN hopping methods. | A: Switch spoofing and double tagging. Q: How does double tagging work? | A: The attacker stacks two 802.1Q tags; the first switch strips only the outer tag, letting the inner tag carry the frame into another VLAN. Q: What command defends against switch spoofing? | A: switchport nonegotiate (plus switchport mode access or switchport mode trunk). Q: What command defends against double tagging? | A: switchport trunk native vlan 999 (move the native VLAN) plus vlan dot1q tag native. Q: What is an STP attack? | A: A rogue switch advertises priority 0 to become the root bridge and sniff all traffic flowing through it. Q: Name the four STP attack defenses. | A: BPDU Guard, Root Guard, Loop Guard, UDLD. Q: What does BPDU Guard do? | A: Disables a port the instant it receives a BPDU, meant for edge ports that should never see one. Q: What does Root Guard do? | A: Blocks a specific port from ever becoming the path to a new root bridge. Q: What does UDLD detect? | A: Unidirectional links, where a link works in only one direction. Q: Name the four DNS poisoning types. | A: Intranet, Internet, Proxy Server, and Cache Poisoning. Q: What makes Intranet DNS spoofing different from the others? | A: It requires the attacker to be on the LAN and pairs with ARP poison routing. Q: What does Internet DNS spoofing change on the victim? | A: The victim's configured DNS server address, via a Trojan. Q: What does Proxy Server DNS poisoning change on the victim? | A: The victim's browser proxy setting, via a Trojan. Q: What makes DNS cache poisoning different from the other three? | A: It corrupts a shared resolver's cache, affecting every later user of that resolver. Q: What is the single most important defense against DNS cache poisoning? | A: DNSSEC (Domain Name System Security Extension). Q: What Wireshark filter shows only Telnet traffic? | A: tcp.port==23 Q: What does the Wireshark operator && mean? | A: Logical AND, combining two required conditions. Q: What tool shows a Google Map of captured public IP locations? | A: OmniPeek. Q: What does Capsa Portable Network Analyzer specialize in? | A: Detecting ARP poisoning and ARP flooding attacks and locating the attack source. Q: Name the three sniffer detection techniques. | A: Ping Method, DNS Method, ARP Method. Q: How does the Ping Method detect a sniffer? | A: Ping with the correct IP but a wrong MAC; only a promiscuous-mode NIC responds. Q: How does the DNS Method detect a sniffer? | A: Watch for unexpected reverse DNS lookup traffic from a host that should not generate it. Q: How does the ARP Method

detect a sniffer? | A: A promiscuous host already has cached ARP info and answers without needing to ARP-probe first. Q: Name the three general sniffer detection approaches. | A: Check devices for promiscuous mode, run an IDS, run network tools.

11. Glossary

ACL (Access Control List): a rule set that restricts network access to a defined, trusted range of addresses. Active sniffing: sniffing a switched network by injecting packets to force the switch to leak traffic. ARP (Address Resolution Protocol): a stateless Layer 2 protocol that resolves IP addresses to MAC addresses. ARP poisoning/spoofing: flooding a target's ARP cache with forged entries so traffic is misdirected to the attacker. ARP-GUARD: a commercial appliance dedicated to detecting ARP spoofing and flooding. arpspoof: a dsniff-suite tool that redirects packets by forging ARP replies. bettercap: a full-featured network attack and monitoring framework. BPDU (Bridge Protocol Data Unit): the control message used by the Spanning Tree Protocol. BPDU Guard: a Cisco feature that disables a port the instant it receives a BPDU. CAM table (Content Addressable Memory table): a switch's fixed-size dynamic table mapping MAC addresses to ports and VLANs. Capsa Portable Network Analyzer: a portable packet capture tool that detects ARP poisoning/flooding and locates the attack source. C2 (Command and Control): the remote server malware or an attacker's beacon reports back to. DAI (Dynamic ARP Inspection): validates ARP packets against the DHCP snooping binding table and drops mismatches. Data Link layer: the OSI layer where sniffers operate, below all layers that carry application data. DHCP (Dynamic Host Configuration Protocol): automatically assigns IP configuration to clients. DHCP snooping: a switch feature that only trusts DHCP server responses arriving on designated trusted ports. DHCP starvation: a denial-of-service attack that exhausts a DHCP server's address pool. DHCPig: a tool used to perform DHCP exhaustion/ starvation attacks. dhcpStarvation.py: a scripted Python tool for DHCP starvation attacks. DNS (Domain Name System): resolves domain names into IP addresses. DNS cache poisoning: forged records injected into a resolver's cache, served to every later requester. DNSSEC (Domain Name System Security Extension): lets a resolver cryptographically verify a DNS response's authenticity. Double tagging: a VLAN hopping method that stacks two 802.1Q tags so a frame crosses into another VLAN. Ettercap: a comprehensive suite for man-in-the-middle attacks, including ARP poisoning. Habu: a hacking toolkit for ARP poisoning, sniffing, and DHCP starvation. Hub: a network device that sends every frame to every port, enabling passive sniffing. Hyenae: a flexible packet generator/flooder tool, used for DHCP starvation. Internet DNS spoofing: a Trojan changes a victim's DNS server address remotely, no LAN access required. Intranet DNS spoofing: LAN-based DNS spoofing paired with ARP poison routing. IP Source Guard: validates traffic's source IP/MAC against the DHCP snooping binding table. IRDP (ICMP Router Discovery Protocol): lets hosts discover routers via ICMP advertisement/solicitation messages. IRDP spoofing: a forged IRDP advertisement that changes a victim's default router. Loop Guard: a Cisco feature

preventing a port from wrongly entering a forwarding state and causing a loop. MAC address: a 48-bit hardware address made of a 3-byte OUI and a 3-byte NIC-specific part. MAC flooding: overflowing a switch's CAM table with fake MAC/IP pairs, forcing it to broadcast like a hub. MAC spoofing/duplicating: reusing a legitimate client's real MAC address to steal their network identity. macof: a dsniff-suite tool that floods a switch's CAM table with random source MAC/IP addresses. MITM (Man-in-the-Middle): an attacker secretly relaying or altering traffic between two parties. NetProbe: an ARP/network monitoring detection tool. netspionage: an ARP/network reconnaissance and detection helper tool. NIC (Network Interface Card): the hardware/driver component that can be set to promiscuous mode. NXDOMAIN: a DNS response meaning the queried domain does not exist; rate-limiting it helps block poisoning attempts. OmniPeek: a packet sniffer that displays a Google Map of captured public IP locations. OpUtils: a ManageEngine tool with ARP spoofing detection capability. OUI (Organizationally Unique Identifier): the first 3 bytes of a MAC address, identifying the manufacturer. Passive sniffing: sniffing via a hub, where no extra packets need to be sent. Port security: a Cisco switch feature that limits how many MAC addresses a port will learn. PRTG Network Monitor: a network performance monitoring dashboard (bandwidth, uptime). Promiscuous mode: a NIC mode that accepts and processes every frame on the wire, not only its own. Proxy server DNS poisoning: a Trojan changes a victim's browser proxy setting to redirect web traffic. RITA (Real Intelligence Threat Analytics): analyzes network traffic for beaconing/command-and-control patterns. Rogue DHCP server: an unauthorized DHCP server that answers requests with bogus configuration. Root Guard: a Cisco feature that blocks a port from ever becoming the path to a new root bridge. SPAN port (Switched Port Analyzer): a switch port configured to mirror all traffic passing through the switch. Switch spoofing: a VLAN hopping method where a rogue switch tricks a legitimate switch into forming a trunk. STP (Spanning Tree Protocol): a Layer 2 protocol that prevents loops by electing a root bridge. STP attack: a rogue switch claims the lowest bridge priority to become the root bridge and sniff all traffic. UDLD (Unidirectional Link Detection): detects and can disable links that only work in one direction. VLAN (Virtual Local Area Network): a logical segmentation boundary on a switch. VLAN hopping: an attack method to bypass VLAN segmentation via switch spoofing or double tagging. Wireshark: a general-purpose packet capture and protocol analysis tool with a display filter language. Yersinia: a multi-protocol Layer 2 attack framework (DHCP, STP, CDP, DTP, and more).

12. Coverage Report

12.1 Map of pages to guide sections

Page(s)	Topic	Guide section(s)
1	Title, learning objectives, network sniffing, promiscuous mode	3.1, 3.2
2	Passive vs active sniffing, Data Link layer	3.3, 3.4
3	SPAN port	3.5
4	MAC address/CAM table, CAM table overflow	3.6, 3.7
5	MAC flooding, MAC attack defense	3.8, 3.9
6	Rogue DHCP server, DHCP starvation	3.10, 3.11
7	DHCP/rogue defense, ARP spoofing	3.12, 3.13
8	ARP spoofing tools, ARP poisoning defense	3.14, 3.15
9	DAI/DHCP snooping config, ARP detection tools	3.16, 3.17
10	MAC spoofing, IRDP spoofing	3.18, 3.19
11	VLAN hopping, STP attack	3.20, 3.21
12	MAC spoofing defense, VLAN hopping defense	3.22, 3.23
13	STP attack defense, DNS poisoning overview	3.24, 3.25
14	Intranet DNS spoofing, Internet DNS spoofing	3.26, 3.27
15	Proxy server DNS poisoning, DNS cache poisoning	3.28, 3.29
16	DNS spoofing defense, Wireshark filters	3.30, 3.31
17	Sniffing tools, sniffing defense (1-7)	3.32, 3.33
18	Sniffing defense (8-15), detecting sniffing	3.33, 3.34
19	Ping/DNS detection methods, ARP detection method	3.35, 3.36

12.2 Items marked [UNCLEAR]

- Page 4: the exact sample MAC address hex values in the CAM table example rows.
- Page 5: the exact macof flood rate/entry count and terminal output text.
- Page 6: Yersinia terminal screenshot details and the Metasploit URL formatting shown in the DHCP starvation tool list.
- Page 8: the exact arpspoof and Habu terminal output values, and the name of the fifth ARP tool icon (read as "larp").
- Page 8/9: the exact hex digits in the DHCP snooping binding table sample MAC address, and the exact values in the sample %SW_DAI-4-DHCP_SNOOPING_DENY syslog message.
- Page 13: the Loop Guard command shown on the slide appears to repeat the BPDU Guard command text; the Command Sheet uses the standard `spanning-tree guard loop` command instead and flags it to verify. Also the exact name of the DNS poisoning tool referenced (read similarly to "DerpNSpoof").
- Page 14: the exact real-website domain/IP in the Intranet DNS Spoofing diagram (clarified using the clearer reading from page 15).

12.3 Items to double check

- Verify the exact macof flooding rate and the precise flag syntax for `macof`, `arpspoof`, and `dsniff` suite tools against your installed version, since the screenshots did not show every flag clearly.
- Verify the Loop Guard IOS command (`spanning-tree guard loop`) against current Cisco documentation, since the source slide's text for Loop Guard appeared identical to its BPDU Guard text.
- Verify current URLs for tools listed from the slides (bettercap, Ettercap, RITM, ARP Spoofer, Habu, OpUtils, netspionage, NetProbe, ARP-GUARD, RITA, Observer Analyzer, PRTG, SolarWinds NPM), since tool websites can move or change over time.
- Verify the exact name of the DNS poisoning tool mentioned on page 13, and the exact name of the fifth ARP tool icon on page 8.

12.4 Coverage confirmation

Every line in the work-file inventory (19 pages, 37 slides/screenshots including repeats) has been placed into exactly one topic section in Section 3, and every named tool, command, flag, protocol, port, acronym, attack, and countermeasure from those pages also appears in Sections 4 through 8. No inventory item was skipped; items that were too small or blurry to read with confidence are explicitly marked [UNCLEAR] above rather than guessed at silently.